Farmacia Hospitalaria

# Carpeta del proyecto

## Sistema de Gestión de Stock — Farmacia Hospitalaria

Documentación técnica: requisitos, casos de uso, arquitectura, datos, API, seguridad, pruebas, instalación, planificación y presupuesto.

Trabajo Final Integrador — Tecnicatura Universitaria en Programación — UTN
Noviembre de 2026

# Índice


1. 1. Relevamiento y requisitos
2. 2. Casos de uso
3. 3. Arquitectura
4. 4. Modelo de datos y diccionario de datos
5. 5. API REST
6. 6. Seguridad
7. 7. Plan de pruebas
8. 8. Instalación y despliegue (manual técnico)
9. 9. Planificación, riesgos y presupuesto
10. 10. Glosario

# 1. Relevamiento y requisitos

## 1.1 Contexto

La farmacia de un hospital público recibe medicamentos de droguerías (proveedores) y los entrega a las áreas del hospital. El control debe hacerse **por lote**, porque cada lote tiene su propia fecha de vencimiento y es la unidad que se rastrea si un laboratorio retira un producto.

### Áreas del hospital (carga inicial)

Guardia, UTI, Pediatría, Neonatología, Quirófano, Internación Clínica, Internación Quirúrgica y Consultorios Externos. Se pueden agregar, renombrar o desactivar desde el sistema.

### Situación anterior

Existía un prototipo con ocho problemas graves (detallados en el informe final, sección 2.1): identidad falsificable, clave del token en el código, stock negativo, hash de contraseña expuesto, registro libre, fechas como texto sin alertas, borrado en cascada y errores sin información ni pruebas. Ninguno se puede repetir.

## 1.2 Partes interesadas y actores

| Parte interesada | Interés |
|---|---|
| Jefatura de farmacia | Que no se venza mercadería, que no falte lo crítico, tener reportes. |
| Farmacéuticos y técnicos | Registrar rápido y sin errores los ingresos y entregas. |
| Áreas del hospital | Recibir lo pedido; tener un comprobante. |
| Dirección del hospital / auditoría | Ver consumo por área y trazabilidad. |
| Área de sistemas | Instalar y mantener el sistema sin costos de licencia. |

| Actor | Rol en el sistema |
|---|---|
| Administrador/a | `ADMIN` |
| Farmacéutico/a | `FARMACEUTICO` |
| Consulta | `CONSULTA` |
| Persona sin sesión | Solo puede ingresar o recuperar su contraseña |

## 1.3 Requisitos funcionales

Prioridad: **A** = imprescindible (lo pide la consigna o el negocio no funciona sin esto), **M** = importante.

### Acceso y usuarios

| Código | Requisito | Prioridad | Rol |
|---|---|---|---|
| RF01 | Iniciar sesión con usuario y contraseña. El usuario no distingue mayúsculas. | A | Todos |
| RF02 | Cerrar sesión. | A | Todos |
| RF03 | Avisar 5 minutos antes de que venza la sesión (8 h). | M | Todos |

| Código | Requisito | Prioridad | Rol |
|---|---|---|---|
| RF04 | Recuperar la contraseña con un código de 6 números de un solo uso, que vence a los 15 minutos (envío simulado). | A | Sin sesión |
| RF05 | Cambiar la contraseña propia indicando la actual. | A | Todos |
| RF06 | Listar usuarios con búsqueda y filtro de activos. | A | ADMIN |
| RF07 | Crear un usuario con nombre de usuario, nombre completo, rol y contraseña inicial. | A | ADMIN |
| RF08 | Editar el nombre completo y el rol de un usuario. | A | ADMIN |
| RF09 | Desactivar y reactivar usuarios (baja lógica). | A | ADMIN |
| RF10 | Restablecer la contraseña de otro usuario. | A | ADMIN |

## Catálogo

| Código | Requisito | Prioridad | Rol |
|---|---|---|---|
| RF11 | Listar, crear, editar, desactivar y reactivar **medicamentos** (código, nombre, presentación, stock mínimo). | A | ADMIN, FARM. (ver: todos) |
| RF12 | Ver el detalle de un medicamento con sus lotes con stock en orden FEFO y su historial de movimientos. | A | Todos |
| RF13 | Listar, crear, editar, desactivar y reactivar **proveedores** (razón social, CUIT, teléfono, correo). | A | ADMIN, FARM. (ver: todos) |
| RF14 | Listar, crear, editar, desactivar y reactivar **áreas**. | A | ADMIN, FARM. (ver: todos) |

## Stock

| Código | Requisito | Prioridad | Rol |
|---|---|---|---|
| RF15 | Registrar el ingreso de un lote: medicamento, proveedor, número de lote, vencimiento y cantidad. | A | ADMIN, FARM. |
| RF16 | Transferir uno o más medicamentos a un área, aplicando FEFO automáticamente. | A | ADMIN, FARM. |
| RF17 | Ver, imprimir y descargar en PDF el comprobante de una transferencia. | A | Todos |
| RF18 | Listar transferencias por rango de fechas y área. | A | Todos |
| RF19 | Listar lotes por estado (vencidos, por vencer, todos con stock). | A | Todos |
| RF20 | Dar de baja un lote vencido indicando el motivo. | A | ADMIN, FARM. |
| RF21 | Ajustar la cantidad de un lote indicando el motivo. | A | ADMIN, FARM. |
| RF22 | Consultar los movimientos de stock (ingreso, egreso, baja, ajuste). | A | Todos |

## Información

| Código | Requisito | Prioridad | Rol |
|---|---|---|---|
| RF23 | Ver alertas: vencidos, sin stock, por vencer (30 días) y stock bajo. | A | Todos |

| Código | Requisito | Prioridad | Rol |
|---|---|---|---|
| RF24 | Ver un panel con indicadores, alertas, últimas 5 transferencias y unidades entregadas por área en los últimos 30 días. | A | Todos |
| RF25 | Reporte de stock de medicamentos (en pantalla, Excel y PDF). | A | Todos |
| RF26 | Reporte de lotes y vencimientos (en pantalla, Excel y PDF). | A | Todos |
| RF27 | Reporte de transferencias a áreas (en pantalla, Excel y PDF). | A | Todos |

## 1.4 Requisitos no funcionales

| Código | Categoría | Requisito | Cómo se verifica |
|---|---|---|---|
| RNF01 | Seguridad | Contraseñas con BCrypt (costo 12), entre 8 y 72 caracteres. | `SeguridadIT`, código de `SeguridadConfig` |
| RNF02 | Seguridad | Sesión con JWT HS256 de 8 horas; clave de al menos 32 bytes por variable de entorno. | `ConfiguracionTest` |
| RNF03 | Seguridad | Permisos validados en el servidor y leídos de la base en cada pedido. | `SeguridadIT` |
| RNF04 | Seguridad | Ninguna respuesta incluye la contraseña ni su hash. | `SeguridadIT` |
| RNF05 | Integridad | El stock nunca es negativo, aun con pedidos simultáneos. | `TransferenciaConcurrenciaIT`, `LoteTest` |
| RNF06 | Integridad | Nada se borra físicamente; las claves foráneas no tienen cascada. | `ProveedorServiceTest`, `V1__esquema.sql` |
| RNF07 | Trazabilidad | Cada cambio de stock genera exactamente un movimiento por lote, con usuario y fecha. La suma de movimientos de un lote es igual a su stock. | `FlujoCompletoIT` |
| RNF08 | Exactitud | "Hoy" se calcula con la zona horaria de Buenos Aires. | `AlertaServiceTest`, `fechas.test.ts` |
| RNF09 | Usabilidad | Textos en español rioplatense; estados con palabra además de color; confirmación de acciones irreversibles. | Revisión, `Insignia.test.tsx` |
| RNF10 | Accesibilidad | WCAG 2.1 AA: contraste, teclado, foco visible, etiquetas, lectores de pantalla. | Playwright + axe en todas las pantallas |
| RNF11 | Rendimiento | Listados paginados en el servidor (20 por página, máximo 100); consultas de stock agregadas en SQL. | `MedicamentoRepositoryIT` |
| RNF12 | Disponibilidad de la interfaz | Uso cómodo a 1366×768 y usable desde 1024 px; menú colapsable en pantallas chicas. | Capturas a 1366×768 |
| RNF13 | Mantenibilidad | Arquitectura en capas, formato automático, pruebas automáticas, decisiones registradas. | `./mvnw verify`, `npm run lint` |
| RNF14 | Portabilidad | Se levanta con Docker Compose en Windows, Linux o macOS. | `docker compose up --build` |
| RNF15 | Costos | Solo software libre y gratuito. | Lista de dependencias |
| RNF16 | Tiempo de respuesta | El frontend corta un pedido a los 15 segundos y avisa. | `cliente.test.ts` |

## 1.5 Reglas de negocio

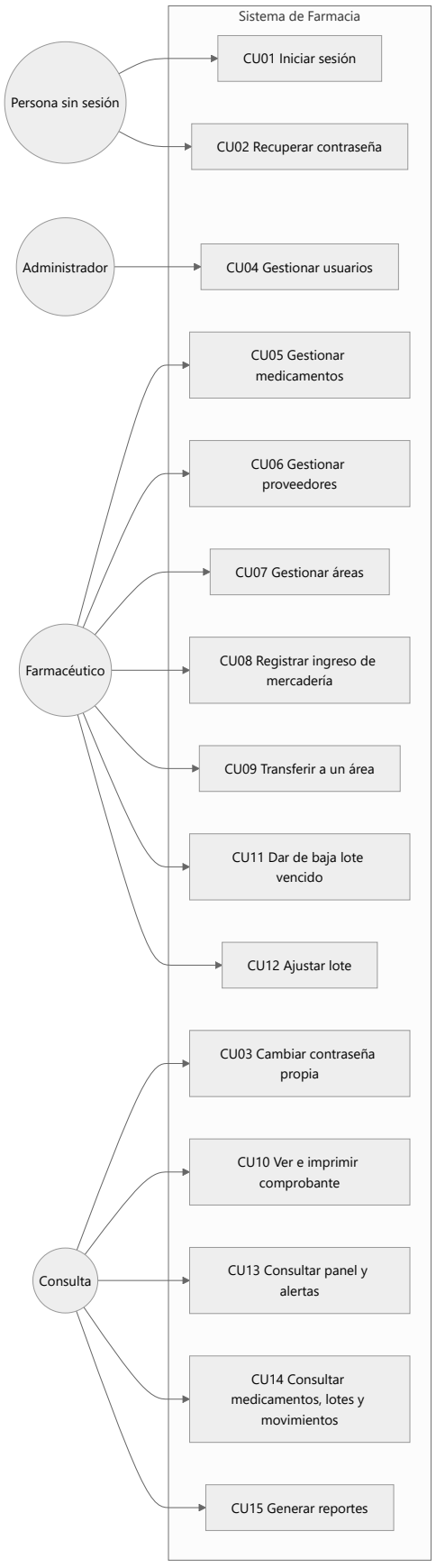
| Código | Regla |
|---|---|
| RN01 | Un lote está **vencido** si su fecha de vencimiento es anterior a hoy. El día del vencimiento todavía se puede usar. |
| RN02 | Un lote está **por vencer** si tiene stock y vence entre hoy y hoy + 30 días (ambos incluidos). El 30 es configurable (`farmacia.alertas.dias-por-vencer`). |
| RN03 | Un lote está **agotado** si su cantidad actual es 0; **vigente** en cualquier otro caso. |
| RN04 | El **stock total** de un medicamento es la suma de las cantidades de sus lotes no vencidos. |
| RN05 | Estado de stock: **Sin stock** (stock 0), **Stock bajo** (0 < stock ≤ mínimo), **OK** (stock > mínimo). Las alertas solo consideran medicamentos activos. |
| RN06 | Un lote vencido con stock genera alerta aunque el medicamento esté inactivo (la mercadería sigue en el depósito). |
| RN07 | El ingreso exige medicamento y proveedor activos, vencimiento ≥ hoy, cantidad > 0 y número de lote no repetido para ese medicamento. |
| RN08 | La transferencia exige área activa, entre 1 y 50 ítems, cantidades > 0 y medicamentos activos. Los ítems repetidos se suman. |
| RN09 | La transferencia aplica **FEFO**: vencimiento ascendente, después fecha de ingreso, después id. |
| RN10 | La transferencia es **todo o nada** e informa todos los faltantes. |
| RN11 | Un lote vencido nunca se transfiere. |
| RN12 | Una transferencia no se modifica ni se anula; un error se corrige con un ajuste. |
| RN13 | Solo se da de baja un lote vencido con stock; la baja deja el lote en 0. Motivo de 5 a 200 caracteres. |
| RN14 | El ajuste fija una cantidad nueva entre 0 y la cantidad inicial, distinta de la actual. Motivo de 5 a 200 caracteres. |
| RN15 | Un medicamento con stock no vencido no se puede desactivar. |
| RN16 | Un proveedor con lotes con stock sí se puede desactivar: sus lotes siguen disponibles, pero no se lo puede elegir en ingresos nuevos. |
| RN17 | Un área inactiva no recibe transferencias. |
| RN18 | El CUIT tiene 11 dígitos y un dígito verificador válido (pesos 5,4,3,2,7,6,5,4,3,2). |
| RN19 | Código de medicamento, CUIT, nombre de usuario y nombre de área son únicos (usuario y área sin distinguir mayúsculas). |
| RN20 | Nadie puede desactivarse a sí mismo. No se puede desactivar ni quitarle el rol al último administrador activo. |
| RN21 | El número de comprobante es `TR-` seguido del id con seis dígitos (`TR-000123`). |
| RN22 | Si no existe ningún administrador al arrancar, se crea el usuario `admin` con la contraseña de `ADMIN_PASSWORD`. |

## 1.6 Supuestos y restricciones

- El sistema se usa desde navegadores de escritorio actuales (Chrome o Edge) dentro de la red del hospital.
- Los usuarios no tienen correo cargado; por eso la recuperación de contraseña es simulada (el código se muestra en pantalla en el modo demostración) y el administrador puede restablecer contraseñas.
- No se registran datos de pacientes.
- La consigna exige: ABM de usuarios, login, roles, recuperación de contraseña, base SQL con modelo ER, PK/FK, integridad y 3FN, arquitectura en capas, manejo de errores, reportes en pantalla y exportables, y una mejora presupuestada. El cumplimiento punto por punto está en `docs/consigna.md`.

# 2. Casos de uso

## 2.1 Diagrama general



El Farmacéutico puede hacer todo lo que hace Consulta, y el Administrador todo lo que hace el Farmacéutico (herencia de actores: cada rol tiene los permisos del anterior más los suyos).

## 2.2 Casos de uso detallados

### CU01 — Iniciar sesión

| | |
|---|---|
| Actor | Persona sin sesión |
| Precondición | Tiene un usuario activo. |
| Disparador | Abre el sistema o su sesión venció. |

**Flujo principal**

1. El sistema muestra el formulario con el foco en "Usuario".
2. La persona ingresa usuario y contraseña y presiona "Ingresar" (o Enter).
3. El sistema muestra "Ingresando…" y deshabilita el botón.
4. El sistema verifica las credenciales, emite un token por 8 horas y muestra el Panel (o la pantalla que la persona quería abrir antes de ingresar).

**Flujos alternativos**

- 4a. Usuario inexistente, inactivo o contraseña incorrecta: el sistema muestra "Usuario o contraseña incorrectos." sin indicar cuál de los tres casos es.
- 1a. Si llegó porque la sesión venció, el sistema muestra "Tu sesión expiró. Volvé a ingresar."

**Postcondición:** sesión iniciada; el menú muestra solo las opciones de su rol.

### CU02 — Recuperar contraseña

| | |
|---|---|
| Actor | Persona sin sesión |
| Precondición | Tiene un usuario activo. |

**Flujo principal**

1. En el ingreso, la persona elige "¿Olvidaste tu contraseña?".
2. Ingresa su nombre de usuario y pide el código.
3. El sistema genera un código de 6 números que vence en 15 minutos, anula los anteriores y "envía" el código (simulado: en el modo demostración lo muestra en pantalla).
4. La persona ingresa el código y la contraseña nueva dos veces.
5. El sistema verifica el código y guarda la contraseña nueva. Muestra "Listo, ya podés ingresar con tu contraseña nueva."

**Flujos alternativos**

- 3a. Si el usuario no existe, el sistema responde exactamente lo mismo (para no revelar qué usuarios existen), pero no genera código.
- 5a. Código incorrecto, vencido o ya usado: error en el campo "Código". Después de 5 intentos fallidos el código se anula; hay que pedir otro ("Pedir otro código").

### CU03 — Cambiar la contraseña propia

Actor: cualquier usuario con sesión. En **Mi cuenta** ingresa la contraseña actual, la nueva y la repetición (8 a 72 caracteres). Si la actual es incorrecta, el error aparece en ese campo.

### CU04 — Gestionar usuarios

| | |
|---|---|
| Actor | Administrador |
| Precondición | Sesión con permiso `USUARIO_GESTIONAR`. |

**Flujo principal (alta)**

1. En **Usuarios** elige "Nuevo usuario".
2. Completa usuario, nombre completo, rol y contraseña inicial.
3. El sistema crea el usuario y lo muestra en la lista.

**Otros flujos**

- **Editar:** cambia nombre completo y rol.
- **Desactivar / reactivar:** con confirmación. No se ofrece sobre el propio usuario.
- **Restablecer contraseña:** ingresa una contraseña nueva para el usuario.

**Excepciones**

- Usuario repetido (sin distinguir mayúsculas): "Ya existe…" (`USERNAME_DUPLICADO`).
- Desactivar o quitar el rol al último administrador activo: rechazado (`ULTIMO_ADMIN`).
- Un usuario desactivado pierde el acceso en su siguiente acción, aunque tenga un token vigente.

### CU05 — Gestionar medicamentos

Actor: Administrador o Farmacéutico (`CATALOGO_EDITAR`).

1. En **Medicamentos** elige "Nuevo medicamento" o "Editar" en el detalle.
2. Completa código, nombre, presentación y stock mínimo.
3. El sistema guarda. El código se normaliza a mayúsculas.

Excepciones: código repetido (`CODIGO_DUPLICADO`); stock mínimo negativo (error de campo); desactivar con stock no vencido (`MEDICAMENTO_CON_STOCK`, el diálogo muestra el motivo).

### CU06 — Gestionar proveedores

Actor: Administrador o Farmacéutico. Alta y edición con razón social, CUIT (con máscara `XX-XXXXXXXX-X`), teléfono y correo. El CUIT se valida con su dígito verificador; repetido → `CUIT_DUPLICADO`. Al desactivar uno con lotes con stock, el sistema avisa: "Tiene N lotes con stock. Van a seguir disponibles, pero no vas a poder elegir este proveedor en ingresos nuevos."

### CU07 — Gestionar áreas

Actor: Administrador o Farmacéutico. Alta y edición del nombre (único sin distinguir mayúsculas). Desactivar y reactivar con confirmación. Un área inactiva no aparece para transferir.

## CU08 — Registrar ingreso de mercadería

| | |
|---|---|
| Actor | Administrador o Farmacéutico ( `STOCK_INGRESAR` ) |
| Precondición | Existen el medicamento y el proveedor, activos. |

**Flujo principal**

1. En **Ingreso de mercadería** (o "Registrar ingreso" desde el detalle de un medicamento), elige el medicamento (buscador por código o nombre) y el proveedor.
2. Ingresa número de lote, fecha de vencimiento y cantidad.
3. El sistema crea el lote, registra un movimiento de **Ingreso** y muestra "Se registró el lote L2345. Stock de : 340."
4. "Registrar otro lote" deja el formulario listo conservando el proveedor.

**Flujos alternativos**

- 2a. Si vence dentro de los próximos 30 días, el sistema avisa ("Podés registrarlo igual.") sin bloquear.
- 3a. Vencimiento anterior a hoy: error en el campo. Lote repetido para ese medicamento: `LOTE_DUPLICADO`. Medicamento o proveedor inactivo: rechazado.

## CU09 — Transferir medicamentos a un área

| | |
|---|---|
| Actor | Administrador o Farmacéutico ( `TRANSFERENCIA_CREAR` ) |
| Precondición | Hay stock no vencido de los medicamentos pedidos. |

**Flujo principal**

1. En **Nueva transferencia** elige el área de destino.
2. Busca un medicamento, indica la cantidad y presiona "Agregar". El sistema muestra el stock disponible y cuánto va a quedar.
3. Repite el paso 2 para cada medicamento. Puede editar cantidades o quitar líneas.
4. Opcionalmente escribe observaciones (hasta 500 caracteres).
5. Presiona "Revisar y confirmar". El sistema muestra un resumen con el stock que va a quedar.
6. Confirma. El sistema muestra "Registrando…", aplica FEFO, descuenta los lotes y muestra el **comprobante**.

**Flujos alternativos**

- 2a. Si agrega un medicamento que ya está en la lista, se suma a su línea: "Se sumó a la línea de ."
- 6a. Si mientras tanto otra persona usó el stock y ya no alcanza ( `STOCK_INSUFICIENTE` ), el sistema cierra el resumen, marca cada línea con problema ("Disponible: X (pediste Y)"), actualiza el stock y **conserva todo lo cargado** para corregir.
- En cualquier momento puede "Descartar" la transferencia (con confirmación).
- Si la sesión vence, el borrador queda guardado en la pestaña y se recupera al volver a ingresar.

**Postcondición:** el stock de los lotes bajó; hay una transferencia con un renglón por lote y un movimiento de **Egreso** por lote.

### CU10 — Ver e imprimir el comprobante

Actor: cualquier usuario. Desde **Transferencias** abre un número `TR-…`. Ve número, fecha y hora, área, quién la hizo, observaciones y el detalle por lote. "Imprimir" muestra solo el comprobante, legible en blanco y negro, con espacio para la firma de quien entrega y quien recibe. También se descarga en PDF.

### CU11 — Dar de baja un lote vencido

Actor: Administrador o Farmacéutico (`STOCK_BAJA_AJUSTE`). En **Lotes y vencimientos** filtra "Vencidos", elige "Dar de baja", escribe el motivo (5 a 200 caracteres) y confirma. El lote queda en 0 y se registra un movimiento de **Baja por vencimiento**. Si el lote no está vencido o ya está en 0, la operación se rechaza.

### CU12 — Ajustar un lote

Actor: Administrador o Farmacéutico. En **Lotes y vencimientos** elige "Ajustar", indica la cantidad nueva (entre 0 y la inicial, distinta de la actual) y el motivo (rotura, diferencia de conteo…). Se registra un movimiento de **Ajuste** con la diferencia (positiva o negativa).

### CU13 — Consultar panel y alertas

Actor: cualquier usuario. El **Panel** muestra cuatro indicadores (Vencidos, Sin stock, Por vencer, Stock bajo) que llevan a la lista filtrada, las alertas agrupadas, las últimas 5 transferencias y las unidades entregadas por área en los últimos 30 días con una barra proporcional.

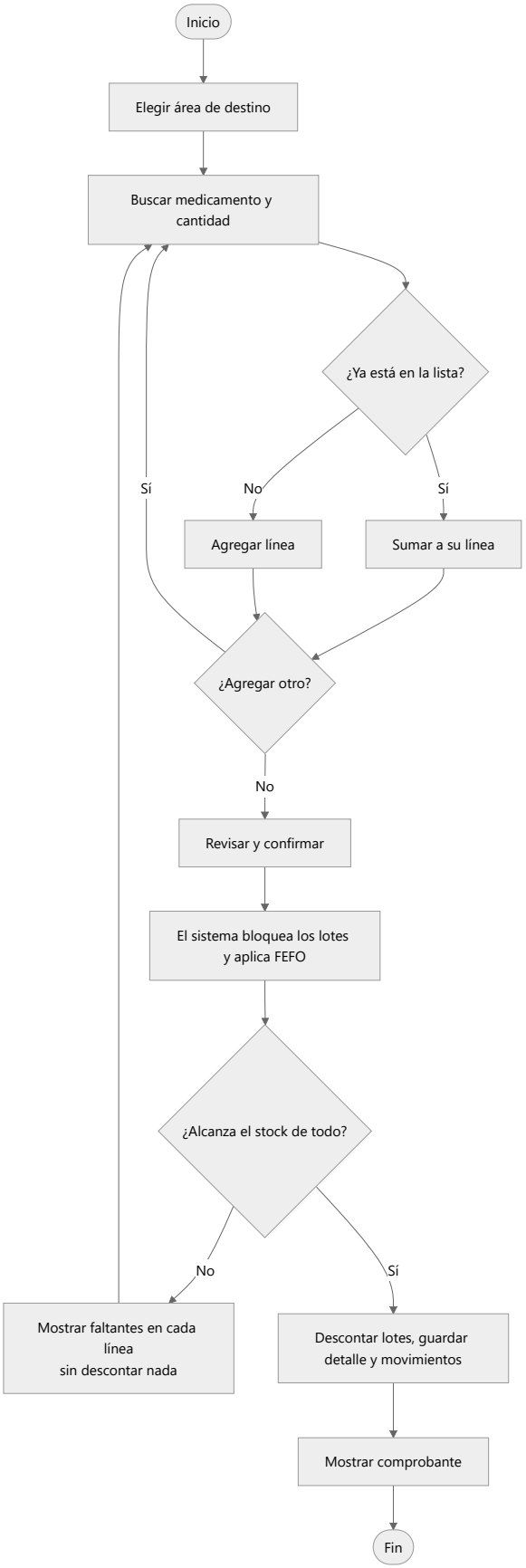
### CU14 — Consultar medicamentos, lotes y movimientos

Actor: cualquier usuario. Listados con búsqueda, filtros, orden y paginación; el detalle de un medicamento muestra sus lotes en orden FEFO y su historial de movimientos.

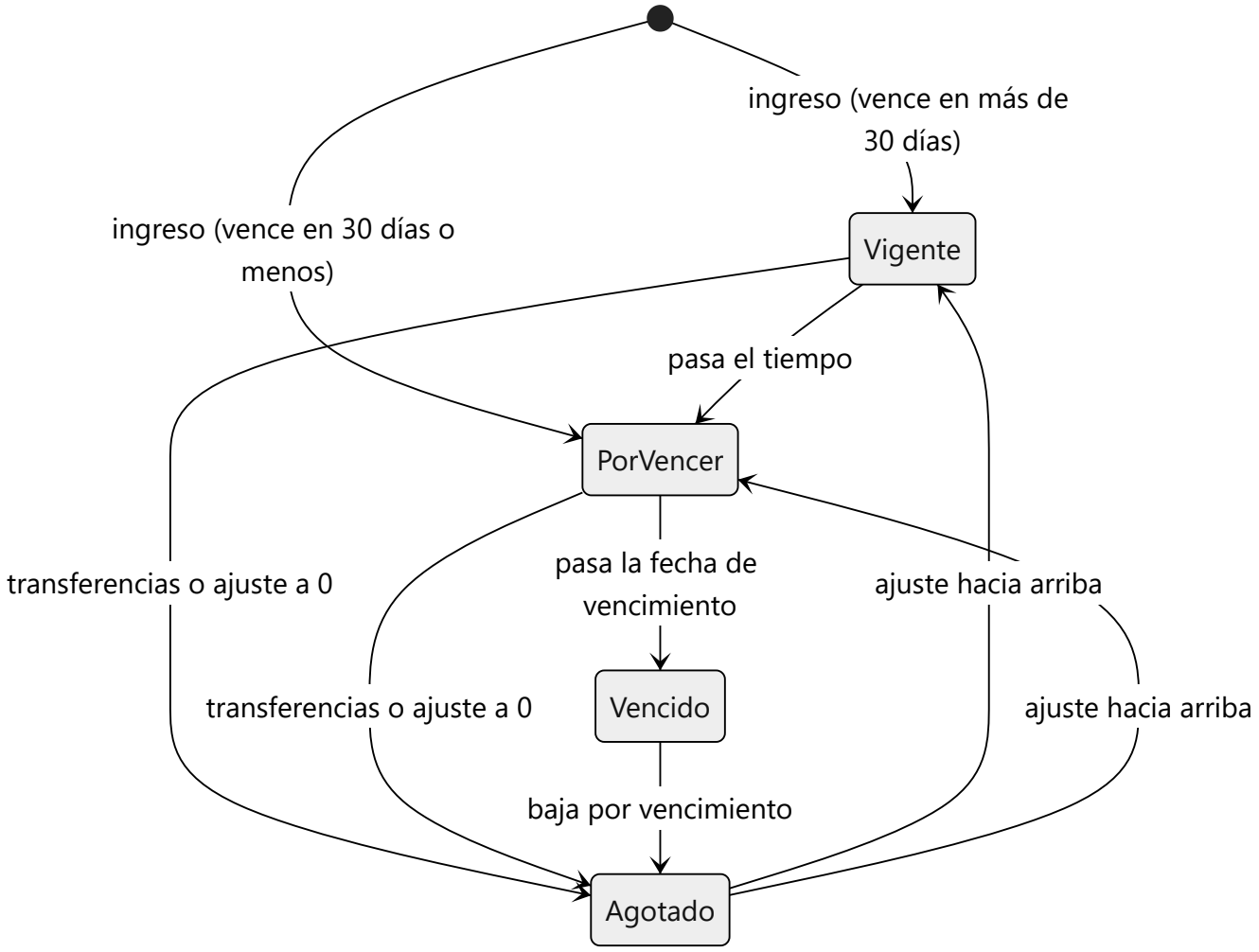
### CU15 — Generar reportes

Actor: cualquier usuario. En **Reportes** elige el reporte (stock de medicamentos, lotes y vencimientos, transferencias a áreas), aplica filtros y elige "Ver en pantalla", "Excel" o "PDF". El reporte incluye todo lo filtrado, no solo una página.

## 2.3 Diagrama de actividad: transferencia



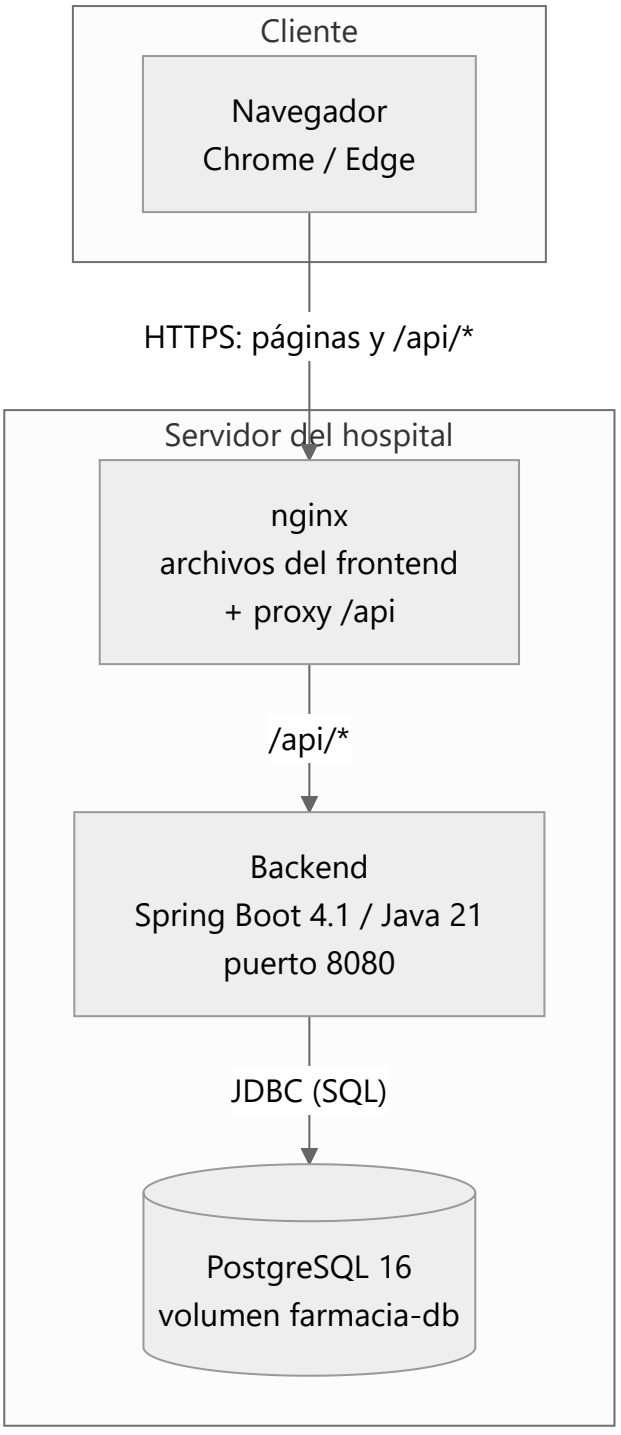
## 2.4 Diagrama de estados de un lote




El estado **no se guarda**: se calcula cada vez a partir de la cantidad actual, la fecha de vencimiento y la fecha de hoy.

# 3. Arquitectura

## 3.1 Vista general




- **Cliente:** una aplicación de una sola página (SPA) en React. Se descarga una vez; después solo pide datos en JSON.
- **nginx:** sirve los archivos compilados del frontend y reenvía `/api/` al backend. Así el navegador ve **un solo origen** y no hace falta CORS. Agrega las cabeceras de seguridad (CSP y otras).
- **Backend:** API REST sin estado (no guarda sesiones: cada pedido trae su token).
- **Base de datos:** PostgreSQL con el esquema versionado por Flyway.

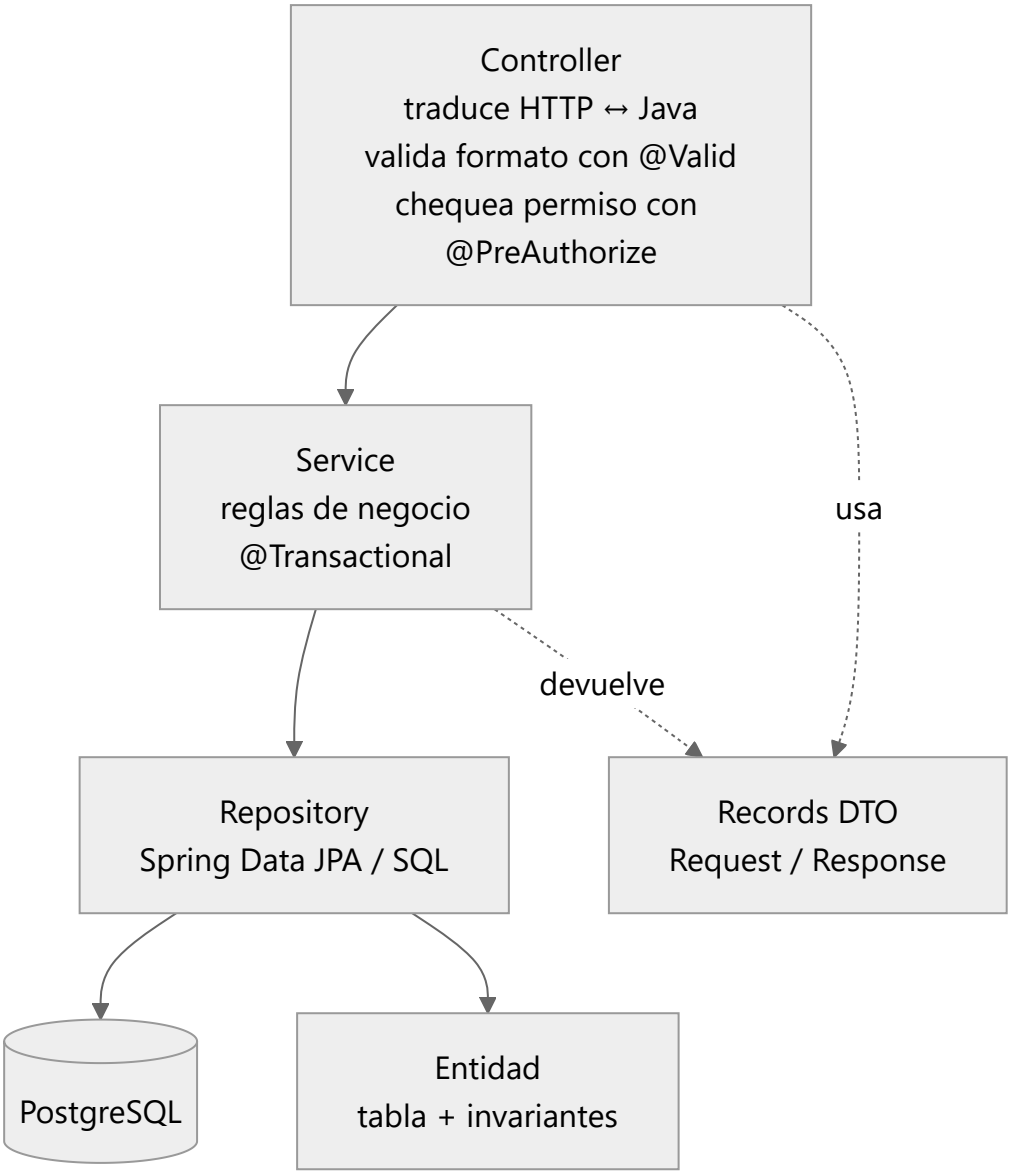
En desarrollo, nginx se reemplaza por el servidor de Vite (`npm run dev`, puerto 5173), que hace el mismo reenvío de `/api`.

## 3.2 Estilo arquitectónico

- **Cliente-servidor** con **API REST**.
- **Monolito modular**: un solo backend organizado por funcionalidad. Se descartaron los microservicios por ser sobreingeniería para una sola farmacia.
- **Arquitectura en capas** dentro de cada módulo.
- **Sin estado (stateless):** la identidad viaja en un JWT; el servidor no guarda sesiones, lo que simplifica y permite reiniciar el servidor sin cerrar las sesiones.

## 3.3 Backend

### Capas



| Capa | Responsabilidad | Prohibido |
|---|---|---|
| Controller | Recibir el pedido, validar el formato, verificar el permiso, devolver el código HTTP. | Lógica de negocio; usar repositorios. |
| Service | Reglas de negocio, coordinación entre entidades, transacciones. | Conocer HTTP. |
| Repository | Consultas a la base. | Reglas de negocio. |
| Entidad | Representar una tabla y proteger sus invariantes (por ejemplo `Lote.descontar`). | Salir por la API. |
| DTO (`record`) | Forma de los datos de entrada y salida. Método `desde(...)` para convertir. | Contener el hash de la contraseña. |

## Paquetes

| Paquete | Contenido principal |
|---|---|
| `auth` | `AuthController`, `AuthService`, `TokenService`, `SeguridadConfig`, `UsuarioJwtConverter`, `Rol`, `Permiso`, `PermisosDeUsuario`, recuperación de contraseña (`RecuperacionService`, `RecuperacionPassword`, `NotificadorRecuperacion`, `CorreoSimulado`), `AdminInicial`. |
| `usuario` | `Usuario`, `UsuarioController`, `UsuarioService`, `UsuarioRepository` y sus DTO. |
| `proveedor` | `Proveedor`, controlador, servicio, repositorio, anotación `@Cuit` y `CuitValidator`. |
| `area` | `Area` y su ABM. |
| `medicamento` | `Medicamento`, consultas de stock calculado (`MedicamentoConsultasImpl` con SQL), `EstadoStock`, reporte de stock. |
| `lote` | `Lote`, `MovimientoStock`, `TipoMovimiento`, `EstadoLote`, ingreso, baja, ajuste, movimientos, reporte de lotes. |
| `transferencia` | `Transferencia`, `DetalleTransferencia`, `TransferenciaService` (FEFO), comprobante PDF, reporte. |
| `alerta` | `AlertaService` (alertas) y `ResumenService` (panel). |
| `common` | `ManejadorErrores`, excepciones de dominio, `RelojConfig` (`Clock`), `Ordenamiento` (lista blanca), `Reporte`, `Exportador`, `Csv`, `ReporteXlsx`, `ReportePdf`, `OpenApiConfig`, `FarmaciaProperties`. |
| `demo` | `DemoDataLoader` (datos de ejemplo), `RelojDemo`, `PerfilesIncompatibles`. |

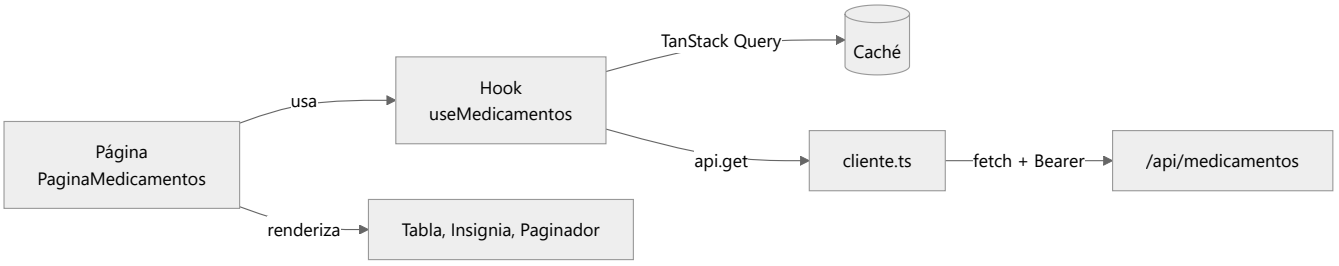
## Perfiles de configuración

| Perfil | Para qué |
|---|---|
| `dev` | Desarrollo local: valores por defecto de conexión, SQL en el log, Swagger activo. |
| `demo` | Carga datos de ejemplo al arrancar y muestra el código de recuperación en pantalla. Swagger activo. |
| `prod` | Producción: Swagger apagado, sin datos de ejemplo. No puede combinarse con `demo` (la aplicación no arranca). |
| `test` | Pruebas automáticas (reloj fijo, base en Testcontainers). |

## 3.4 Frontend

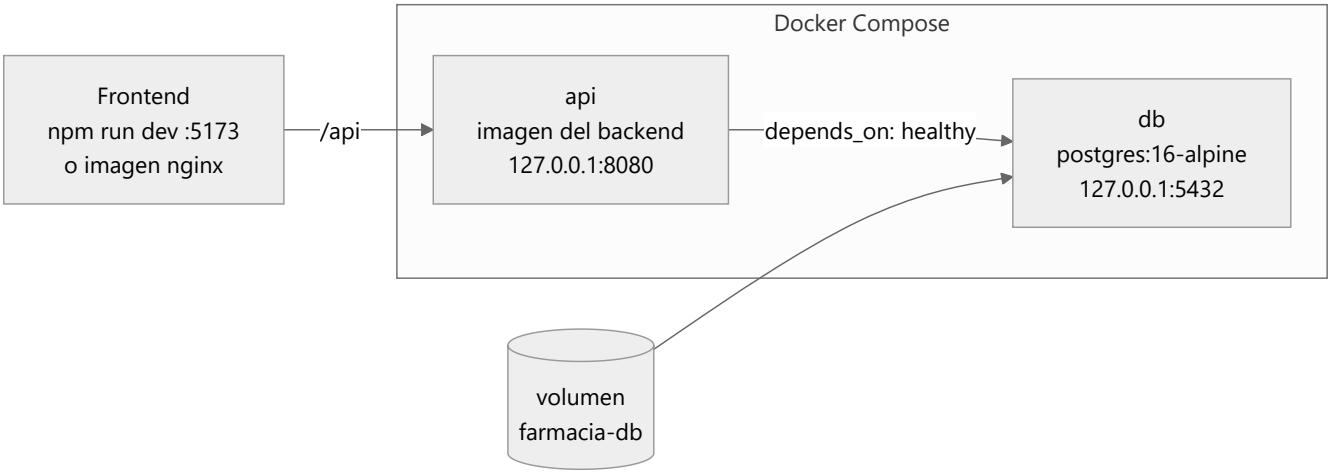
```
frontend/src/
├── main.tsx            punto de entrada: monta <App/> e importa los estilos
├── App.tsx             rutas (React Router) y proveedores (sesión, consultas, avisos)
├── api/                comunicación con el backend
│   ├── cliente.ts      ÚNICO lugar con fetch: token, errores, tiempo máximo
│   ├── errores.ts      lectura del ProblemDetail y mensajes
│   ├── consultas.ts    configuración de TanStack Query
│   ├── esquema.ts      tipos generados del OpenAPI (no se edita a mano)
│   ├── tipos.ts        nombres cortos de esos tipos
│   └── <recurso>.ts    hooks por recurso: medicamentos, lotes, transferencias…
├── auth/               sesión, ingreso, recuperación, rutas protegidas
├── components/         piezas reutilizables: Tabla, Modal, Avisos, Menu, Layout, Insignia…
├── hooks/              useFormulario, useFiltrosUrl, useDebounce, useDescarga
├── pages/              una carpeta por pantalla
├── utils/              fechas, números, CUIT, estado de stock, validaciones
└── styles/             tokens (colores, espacios), base, componentes, animaciones, impresión
```

Flujo de datos de una pantalla:




| Decisión | Motivo |
|---|---|
| Sin librerías de componentes ni de estilos (sin Bootstrap/Tailwind) | Control total del diseño y la accesibilidad; menos dependencias. |
| TanStack Query para datos del servidor | Caché, carga, error e invalidación en un solo lugar. |
| Tipos generados del OpenAPI del backend | Si el contrato cambia, el frontend no compila: el error aparece antes de llegar al usuario. |
| Filtros en la URL | Recargar, compartir el enlace o volver atrás conserva la vista. |
| `<dialog>` nativo para los modales | Bloquea el fondo, cierra con Escape y maneja el foco sin librerías. |

## 3.5 Vista de despliegue




- Los puertos se publican solo en `127.0.0.1` (la máquina local). Para exponer el sistema en la red del hospital se pone delante el nginx del frontend (o un proxy del hospital con HTTPS).
- El backend espera a que la base esté "sana" (`pg_isready`) antes de arrancar.
- Los datos viven en el volumen `farmacia-db` y sobreviven a reinicios de los contenedores.

## 3.6 Integración continua

`.github/workflows/ci.yml` corre en cada `push` y *pull request*: instala Java 21, ejecuta `./mvnw -B verify` (formato, pruebas unitarias y de integración con PostgreSQL en Testcontainers) y guarda el reporte de cobertura de JaCoCo.

## 3.7 Decisiones de arquitectura

El registro completo, con alternativas y motivos, está en `docs/decisiones.md` y `docs/decisiones-frontend.md`. Las más importantes:

| Decisión | Alternativa descartada | Por qué |
|---|---|---|
| Spring Boot 4.1 | Spring Boot 3.5 | La 3.5 perdió el soporte gratuito en junio de 2026. |
| JWT con el soporte oficial de Spring Security | Librería JWT externa o filtro escrito a mano | Menos código propio; Spring valida firma, vencimiento y emisor. |
| Permisos leídos de la base en cada pedido | Rol dentro del token | Una baja o un cambio de rol tienen efecto inmediato. |
| Bajas lógicas (`activo = false`) | `DELETE` | No se pierde historial; se puede reactivar. |
| Stock calculado | Columna `stock_total` | Nunca se contradice con los lotes (3FN). |
| Bloqueo pesimista ordenado por id | Bloqueo optimista con reintentos | Con stock escaso el optimista fallaría seguido; el orden fijo evita deadlocks. |
| Monorepo | Repositorios separados | Un `docker compose up` levanta todo; un solo historial. |

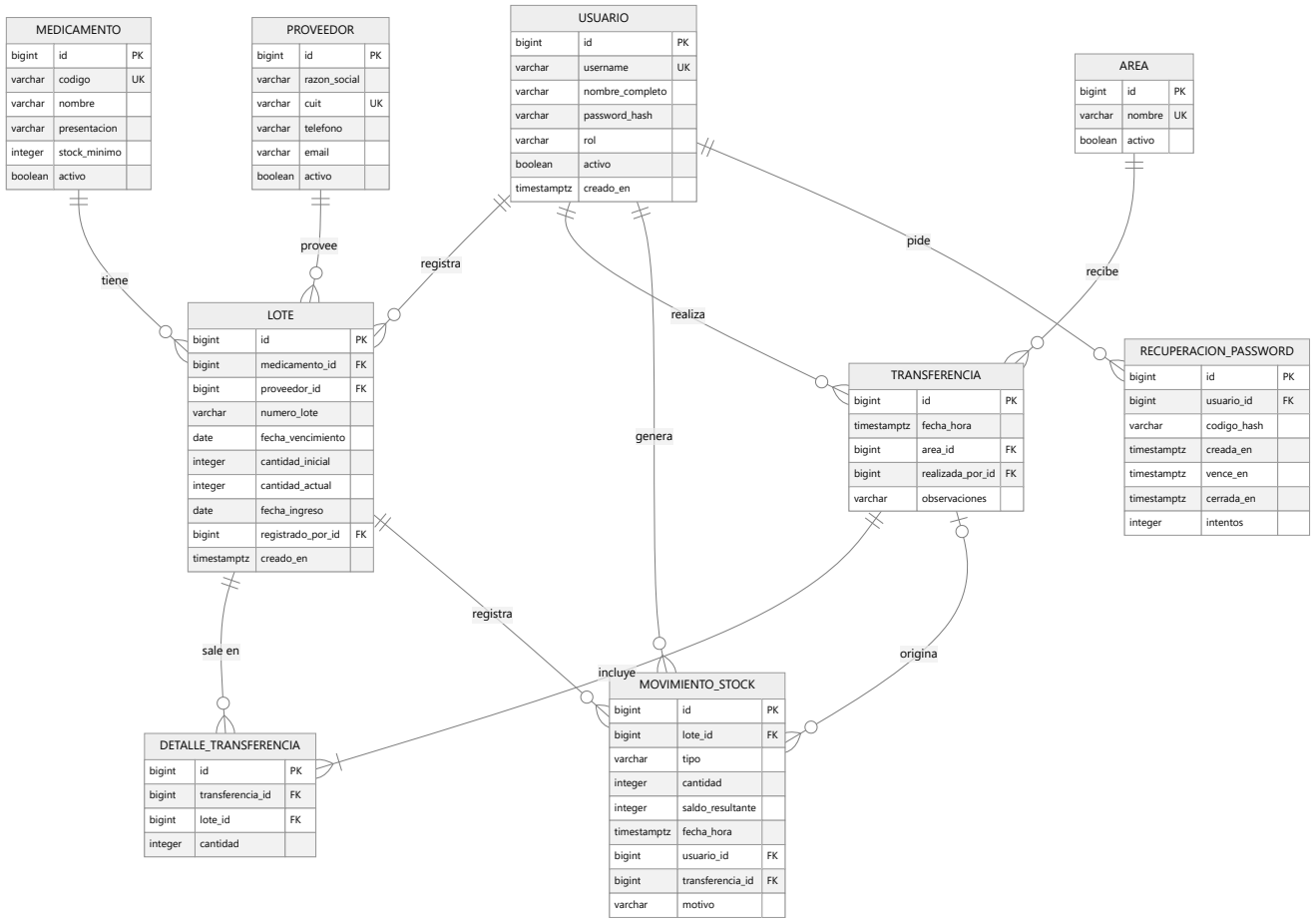
# 4. Modelo de datos y diccionario de datos

Base: **PostgreSQL 16**. El esquema lo crean las migraciones de Flyway en `backend/src/main/resources/db/migration/`; Hibernate solo **valida** que las clases coincidan con las tablas (`ddl-auto: validate`), nunca las modifica.

| Migración | Contenido |
|---|---|
| `V1__esquema.sql` | Las ocho tablas del negocio con claves, restricciones e índices. |
| `V2__datos_base.sql` | Las ocho áreas iniciales del hospital. |
| `V3__recuperacion_password.sql` | Tabla de pedidos de recuperación de contraseña. |

Una migración aplicada **no se edita nunca**: cualquier cambio va en una versión nueva (`V4__...`).

## 4.1 Diagrama entidad-relación




**Cómo leer las patas del diagrama:** `||` = exactamente uno; `o{` = cero o muchos; `|{` = uno o muchos; `|o` = cero o uno. Por ejemplo, `MEDICAMENTO ||--o{ LOTE` se lee "un medicamento tiene cero o muchos lotes; cada lote es de exactamente un medicamento".

## 4.2 Diccionario de datos

Convenciones: **PK** clave primaria, **FK** clave foránea, **UK** valor único, **NN** no nulo. Todas las PK son `bigint generated by default as identity` (número que la base asigna sola).

### usuario

Personas que usan el sistema.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | Identificador interno. |
| username | varchar(50) | NN, UK `uq_usuario_username` | Nombre de usuario, guardado en minúsculas. |
| nombre_completo | varchar(100) | NN | Nombre y apellido. |
| password_hash | varchar(100) | NN | Hash BCrypt de la contraseña. Nunca sale por la API. |
| rol | varchar(20) | NN, CHECK `ck_usuario_rol` ∈ {ADMIN, FARMACEUTICO, CONSULTA} | Rol que define los permisos. |
| activo | boolean | NN, por defecto `true` | Baja lógica. |
| creado_en | timestamptz | NN | Fecha y hora de alta. |

### proveedor

Droguerías o laboratorios que entregan mercadería.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | |
| razon_social | varchar(150) | NN | Nombre legal. |
| cuit | varchar(11) | NN, UK `uq_proveedor_cuit`, CHECK `ck_proveedor_cuit` (11 dígitos) | CUIT sin guiones. El dígito verificador se valida en el código. |
| telefono | varchar(30) | opcional | |
| email | varchar(120) | opcional | Validado con formato de correo. |
| activo | boolean | NN, por defecto `true` | Baja lógica. |

### area

Servicios del hospital que reciben medicamentos.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | |
| nombre | varchar(80) | NN, índice único `ux_area_nombre` sobre `lower(nombre)` | Único sin distinguir mayúsculas ("UTI" = "uti"). |
| activo | boolean | NN, por defecto `true` | Un área inactiva no recibe transferencias. |

### medicamento

Catálogo. **No guarda el stock**: se calcula sumando sus lotes.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | |
| codigo | varchar(30) | NN, UK `uq_medicamento_codigo` | Código interno, en mayúsculas ( `AMOX-500` ). |
| nombre | varchar(120) | NN | Nombre genérico ( `Amoxicilina` ). |
| presentacion | varchar(120) | NN | Forma y concentración ( `comprimidos 500 mg` ). |
| stock_minimo | integer | NN, CHECK `ck_medicamento_stock_minimo` (≥ 0) | Umbral para la alerta de stock bajo. |
| activo | boolean | NN, por defecto `true` | Baja lógica (solo sin stock no vencido). |

### lote

Cada partida recibida de un medicamento.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | |
| medicamento_id | bigint | NN, FK → medicamento | De qué medicamento es. |
| proveedor_id | bigint | NN, FK → proveedor | Quién lo entregó. |
| numero_lote | varchar(50) | NN; UK compuesta `uq_lote_medicamento_numero` (medicamento_id, numero_lote) | Número impreso en el envase. |
| fecha_vencimiento | date | NN | Último día en que se puede usar. |
| cantidad_inicial | integer | NN, CHECK `ck_lote_cantidad_inicial` (> 0) | Unidades recibidas. |
| cantidad_actual | integer | NN, CHECK `ck_lote_cantidad_actual` (0 ≤ actual ≤ inicial) | Unidades que quedan. |
| fecha_ingreso | date | NN | Día del ingreso (de Argentina). |
| registrado_por_id | bigint | NN, FK → usuario | Quién cargó el ingreso. |
| creado_en | timestamptz | NN | Instante exacto del registro. |

Índices: `(medicamento_id, fecha_vencimiento)` para el FEFO; `(fecha_vencimiento) WHERE cantidad_actual > 0` (índice **parcial**: solo lotes con stock, que son los que interesan para las alertas); `(proveedor_id)` .

### transferencia

Cabecera de una entrega a un área. Inmutable.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | El número visible `TR-000123` se deriva de este id. |
| fecha_hora | timestamptz | NN | Momento de la entrega. |

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| area_id | bigint | NN, FK → area | Área que recibe. |
| realizada_por_id | bigint | NN, FK → usuario | Quién entregó (sale del token). |
| observaciones | varchar(500) | opcional | Texto libre. |

Índices: `(fecha_hora)`, `(area_id)`.

### detalle_transferencia

Renglón del comprobante: cuántas unidades salieron de **cada lote**.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | |
| transferencia_id | bigint | NN, FK → transferencia | |
| lote_id | bigint | NN, FK → lote | |
| cantidad | integer | NN, CHECK `ck_detalle_cantidad` (> 0) | |

Si se piden 150 unidades de un medicamento y el FEFO toma 100 de un lote y 50 de otro, hay **dos** renglones.

### movimiento_stock

Registro de auditoría: cada cambio de stock de cada lote.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | |
| lote_id | bigint | NN, FK → lote | |
| tipo | varchar(20) | NN, CHECK `ck_movimiento_tipo` ∈ {INGRESO, EGRESO, BAJA_VENCIMIENTO, AJUSTE} | |
| cantidad | integer | NN, CHECK `ck_movimiento_cantidad` (≠ 0) | Variación **con signo**: +100 en un ingreso, −20 en un egreso. |
| saldo_resultante | integer | NN, CHECK `ck_movimiento_saldo` (≥ 0) | Stock del lote después del movimiento. |
| fecha_hora | timestamptz | NN | |
| usuario_id | bigint | NN, FK → usuario | Quién lo generó. |
| transferencia_id | bigint | FK → transferencia, opcional | Solo en los EGRESO. |
| motivo | varchar(200) | opcional | Obligatorio (en el código) para BAJA_VENCIMIENTO y AJUSTE. |

**Invariante:** la suma de `cantidad` de los movimientos de un lote es igual a su `cantidad_actual`. La prueba `FlujoCompletoIT` lo verifica.

| Tipo | Signo | Cuándo |
|---|---|---|
| INGRESO | + | Al registrar el lote (igual a la cantidad inicial). |
| EGRESO | − | En cada transferencia, uno por lote tocado. |

| Tipo | Signo | Cuándo |
|---|---|---|
| BAJA_VENCIMIENTO | − | Baja de un lote vencido (toda su cantidad). |
| AJUSTE | ± | Corrección de inventario (diferencia entre la cantidad nueva y la anterior). |

### recuperacion_password

Pedidos de código para recuperar la contraseña.

| Columna | Tipo | Restricciones | Descripción |
|---|---|---|---|
| id | bigint | PK | |
| usuario_id | bigint | NN, FK → usuario | |
| codigo_hash | varchar(100) | NN | Hash BCrypt del código (nunca el código en texto). |
| creada_en | timestamptz | NN | |
| vence_en | timestamptz | NN, CHECK `ck_recuperacion_vencimiento` (> creada_en) | 15 minutos después. |
| cerrada_en | timestamptz | opcional | Se completa al usarse, anularse o agotar los intentos. |
| intentos | integer | NN, por defecto 0, CHECK `ck_recuperacion_intentos` (≥ 0) | Intentos fallidos (máximo 5). |

Índice parcial: `(usuario_id) WHERE cerrada_en IS NULL` (solo los pedidos abiertos).

## 4.3 Relaciones y cardinalidades

| Clave foránea | Relación | Cardinalidad |
|---|---|---|
| `lote.medicamento_id → medicamento.id` | El lote es de un medicamento. | 1 : N |
| `lote.proveedor_id → proveedor.id` | El lote lo entregó un proveedor. | 1 : N |
| `lote.registrado_por_id → usuario.id` | Quién cargó el ingreso. | 1 : N |
| `transferencia.area_id → area.id` | Área que recibe. | 1 : N |
| `transferencia.realizada_por_id → usuario.id` | Quién entregó. | 1 : N |
| `detalle_transferencia.transferencia_id → transferencia.id` | Renglón del comprobante. | 1 : 1..N |
| `detalle_transferencia.lote_id → lote.id` | De qué lote salió. | 1 : N |
| `movimiento_stock.lote_id → lote.id` | Movimiento del lote. | 1 : N |
| `movimiento_stock.usuario_id → usuario.id` | Quién lo generó. | 1 : N |
| `movimiento_stock.transferencia_id → transferencia.id` | Transferencia que originó un egreso. | 0..1 : N |
| `recuperacion_password.usuario_id → usuario.id` | Pedido de código. | 1 : N |

`detalle_transferencia` es una **tabla intermedia** que resuelve la relación **muchos a muchos** entre `transferencia` y `lote`: una transferencia toma varios lotes y un lote se reparte en varias transferencias.

Ninguna clave foránea tiene `ON DELETE CASCADE`. Si alguien intentara borrar un medicamento con lotes, la base lo impediría. Las bajas son lógicas.

## 4.4 Integridad en tres capas

| Regla | Base de datos (última barrera) | Entidad Java | DTO de entrada |
|---|---|---|---|
| Stock entre 0 y la cantidad inicial | `ck_lote_cantidad_actual` | `Lote.descontar`, `Lote.ajustar` lanzan excepción | `@Positive` en cantidades |
| Ingreso con al menos 1 unidad | `ck_lote_cantidad_inicial` | Constructor de `Lote` | `@Positive` en `IngresoRequest` |
| Lote no repetido por medicamento | `uq_lote_medicamento_numero` | — | `LoteService` responde 409 `LOTE_DUPLICADO` |
| Únicos (usuario, CUIT, código, área) | `UNIQUE` / índice único | — | Servicios controlan antes; `ManejadorErrores` traduce la violación si llegan dos pedidos juntos |
| CUIT de 11 dígitos | `ck_proveedor_cuit` | — | `@Cuit` (también el dígito verificador) |
| Roles y tipos válidos | `ck_usuario_rol`, `ck_movimiento_tipo` | Enums `Rol`, `TipoMovimiento` | — |
| Stock mínimo ≥ 0 | `ck_medicamento_stock_minimo` | — | `@PositiveOrZero` |

## 4.5 Normalización

**1FN — valores atómicos, sin grupos repetidos.** Cada celda tiene un solo valor. Los renglones de una transferencia no son columnas `medicamento1`, `medicamento2`… sino filas de `detalle_transferencia`. Los movimientos de un lote son filas de `movimiento_stock`.

**2FN — sin dependencias parciales.** Todas las claves primarias son simples (`id`), así que ningún atributo puede depender de "parte" de la clave. En `lote`, cuya clave natural es compuesta (medicamento + número de lote), el vencimiento y las cantidades dependen del lote entero, no del medicamento solo.

**3FN — sin dependencias transitivas.** Ningún atributo no clave depende de otro atributo no clave:

| Dato que NO se guarda | Por qué | Cómo se obtiene |
|---|---|---|
| Nombre del medicamento en `lote` | Dependería de `medicamento_id`, no del lote. | JOIN con `medicamento`. |
| Razón social del proveedor en `lote` | Ídem con `proveedor_id`. | JOIN. |
| Nombre del área o del usuario en `transferencia` | Ídem. | JOIN. |
| Stock total del medicamento | Depende de los lotes; se desactualizaría. | `SUM(cantidad_actual)` de lotes no vencidos. |
| Estado del lote (vigente, por vencer…) | Depende de la fecha de hoy. | Se calcula al consultar. |
| Total de unidades de una transferencia | Depende de los detalles. | `SUM(detalle.cantidad)`. |
| Número `TR-000123` | Depende del `id`. | Se formatea desde el id. |

**Redundancia controlada:** `movimiento_stock.saldo_resultante` se podría calcular sumando los movimientos anteriores, pero se guarda a propósito: es el saldo que vio el usuario en ese momento, es un dato histórico que nunca se actualiza (no genera anomalías) y permite detectar inconsistencias.

## 4.6 Consultas representativas

Stock total y próximo vencimiento por medicamento (de `MedicamentoConsultasImpl`). La subconsulta agrupa los lotes con stock y no vencidos por medicamento; el `left join` conserva los medicamentos que no tienen ninguno (stock 0):

```
select m.id, m.codigo, m.nombre, m.presentacion, m.stock_minimo, m.activo,
       coalesce(s.stock, 0) as stock_total, s.proximo as proximo_vencimiento
from medicamento m
left join (
    select l.medicamento_id,
           sum(l.cantidad_actual) as stock,
           min(l.fecha_vencimiento) as proximo
    from lote l
    where l.cantidad_actual > 0 and l.fecha_vencimiento >= :hoy
    group by l.medicamento_id
) s on s.medicamento_id = m.id
-- + filtros (búsqueda, estado, activo) + order by de una lista blanca + limit/offset
```

Lotes candidatos de una transferencia, bloqueados. En el código es una consulta JPQL con `@Lock(PESSIMISTIC_WRITE)` (`LoteRepository.bloquearDisponibles`); Hibernate la traduce a este SQL:

```
select * from lote
where medicamento_id in (:ids) and cantidad_actual > 0 and fecha_vencimiento >= :hoy
order by id
for update;
```

# 5. API REST

La documentación interactiva está en **Swagger UI** (perfiles `dev` y `demo`): http://localhost:8080/swagger-ui.html. El contrato en formato OpenAPI está en http://localhost:8080/v3/api-docs. Hay pedidos listos para probar con la extensión *REST Client* de VS Code en `docs/api/farmacia.http`.

## 5.1 Convenciones

| Tema | Convención |
|---|---|
| Prefijo | Todas las rutas empiezan con `/api`. |
| Formato | JSON con nombres en camelCase. |
| Fechas | Día: `2026-11-30`. Instante: `2026-11-09T17:49:00-03:00`. |
| Autenticación | Cabecera `Authorization: Bearer <token>` en todo salvo login y recuperación. |
| Paginación | `page` (desde 0), `size` (20 por defecto, máximo 100), `sort=campo,asc` o `campo,desc`. Respuesta: `{ "content": [...], "page": { "number", "size", "totalElements", "totalPages" } }`. |
| Orden | Solo los campos permitidos de cada listado; otro → 400 `ORDEN_INVALIDO`. |
| Filtro `activo` | Opcional. Sin él, vienen activos e inactivos. |
| Respuestas | 200 consulta o edición; 201 creación (con cabecera `Location`); 202 aceptado (pedido de código); 204 sin cuerpo (cambios de estado y de contraseña). |
| Bajas | `PATCH /{id}/estado` con `{ "activo": false }` (o `true` para reactivar). No hay `DELETE`. |

## 5.2 Endpoints

Permiso: el que exige cada ruta (ver la matriz en 06-seguridad.md).

### Autenticación

| Método y ruta | Permiso | Entrada | Respuesta |
|---|---|---|---|
| `POST /api/auth/login` | público | `{username, password}` | 200 `{token, expiraEn, usuario}` |
| `GET /api/auth/me` | con sesión | — | 200 `{id, username, nombreCompleto, rol, permisos[]}` |
| `PATCH /api/auth/me/password` | con sesión | `{passwordActual, passwordNueva}` | 204 |
| `POST /api/auth/recuperacion` | público | `{username}` | 202 `{mensaje, codigoSimulado}` (`codigoSimulado` solo en demo) |
| `POST /api/auth/recuperacion/confirmar` | público | `{username, codigo, passwordNueva}` | 204 |

### Usuarios

| Método y ruta | Permiso | Entrada | Respuesta |
|---|---|---|---|
| `GET /api/usuarios` | USUARIO_GESTIONAR | `q`, `activo`; orden: `username`, `nombreCompleto`, `creadoEn` | Página de usuarios |

| Método y ruta | Permiso | Entrada | Respuesta |
|---|---|---|---|
| `POST /api/usuarios` | USUARIO_GESTIONAR | `{username, nombreCompleto, rol, password}` | 201 |
| `PUT /api/usuarios/{id}` | USUARIO_GESTIONAR | `{nombreCompleto, rol}` | 200 |
| `PATCH /api/usuarios/{id}/estado` | USUARIO_GESTIONAR | `{activo}` | 204 |
| `PATCH /api/usuarios/{id}/password` | USUARIO_GESTIONAR | `{passwordNueva}` | 204 |

## Proveedores y áreas

| Método y ruta | Permiso | Entrada | Respuesta |
|---|---|---|---|
| `GET /api/proveedores` | LECTURA | `q` (razón social o CUIT), `activo`; orden: `razonSocial` | Página con `lotesConStock` y CUIT con guiones |
| `GET /api/proveedores/{id}` | LECTURA | — | 200 |
| `POST /api/proveedores` | CATALOGO_EDITAR | `{razonSocial, cuit, telefono, email}` | 201 |
| `PUT /api/proveedores/{id}` | CATALOGO_EDITAR | ídem | 200 |
| `PATCH /api/proveedores/{id}/estado` | CATALOGO_EDITAR | `{activo}` | 204 |
| `GET /api/areas` | LECTURA | `q`, `activo`; orden: `nombre` | Página |
| `POST /api/areas` · `PUT /api/areas/{id}` | CATALOGO_EDITAR | `{nombre}` | 201 / 200 |
| `PATCH /api/areas/{id}/estado` | CATALOGO_EDITAR | `{activo}` | 204 |

## Medicamentos

| Método y ruta | Permiso | Entrada | Respuesta |
|---|---|---|---|
| `GET /api/medicamentos` | LECTURA | `q` (código o nombre), `estado` (`SIN_STOCK`, `STOCK_BAJO`, `OK`), `activo`; orden: `codigo`, `nombre`, `stockTotal`, `proximoVencimiento` | Página con `stockTotal`, `estadoStock`, `proximoVencimiento` |
| `GET /api/medicamentos/exportar` | LECTURA | mismos filtros + `formato=CSV\|XLSX\|PDF` | Archivo `stock_medicamentos.*` |
| `GET /api/medicamentos/{id}` | LECTURA | — | Medicamento + `lotes` con stock en orden FEFO |
| `POST /api/medicamentos` | CATALOGO_EDITAR | `{codigo, nombre, presentacion, stockMinimo}` | 201 |
| `PUT /api/medicamentos/{id}` | CATALOGO_EDITAR | ídem | 200 |
| `PATCH /api/medicamentos/{id}/estado` | CATALOGO_EDITAR | `{activo}` | 204 (409 `MEDICAMENTO_CON_STOCK`) |

## Lotes y movimientos

| Método y ruta | Permiso | Entrada | Respuesta |
|---|---|---|---|
| `POST /api/medicamentos/{id}/lotes` | STOCK_INGRESAR | `{proveedorId, numeroLote, fechaVencimiento, cantidad}` | 201 `{lote, stockTotalResultante}` |
| `GET /api/lotes` | LECTURA | `medicamentoId`, `estado` (uno o varios: `VIGENTE`, `POR_VENCER`, `VENCIDO`, `AGOTADO`); orden: `fechaVencimiento`, `fechaIngreso` | Página con `estado` y `diasParaVencer` |
| `GET /api/lotes/exportar` | LECTURA | mismos filtros + `formato` | Archivo `lotes_vencimientos.*` |
| `POST /api/lotes/{id}/baja` | STOCK_BAJA_AJUSTE | `{motivo}` | 200 lote |
| `POST /api/lotes/{id}/ajuste` | STOCK_BAJA_AJUSTE | `{cantidadNueva, motivo}` | 200 lote |
| `GET /api/movimientos` | LECTURA | `loteId`, `medicamentoId`, `tipo`, `desde`, `hasta`; orden: `fechaHora` (desc) | Página |

## Transferencias

| Método y ruta | Permiso | Entrada | Respuesta |
|---|---|---|---|
| `POST /api/transferencias` | TRANSFERENCIA_CREAR | `{areaId, observaciones, items: [{medicamentoId, cantidad}]}` | 201 comprobante completo |
| `GET /api/transferencias` | LECTURA | `desde`, `hasta` (inclusive), `areaId`; orden: `fechaHora` (desc) | Página con `numero`, `cantidadItems`, `unidadesTotales` |
| `GET /api/transferencias/{id}` | LECTURA | — | Comprobante |
| `GET /api/transferencias/{id}/pdf` | LECTURA | — | PDF del comprobante con lugar para firmas |
| `GET /api/transferencias/exportar` | LECTURA | `desde` y `hasta` obligatorios (máx. 366 días), `areaId`, `formato` | Archivo |

## Alertas y panel

| Método y ruta | Permiso | Respuesta |
|---|---|---|
| `GET /api/alertas` | LECTURA | `{diasPorVencer, sinStock[], stockBajo[], vencidos[], porVencer[]}` |
| `GET /api/resumen` | LECTURA | `{totales, diasPorVencer, medicamentosActivos, transferenciasHoy, ultimasTransferencias[5], egresosPorArea[]}` |

## Infraestructura

| Ruta | Para qué |
|---|---|
| `GET /actuator/health` | Estado del servicio (`UP`), sin detalles. Público. |
| `/swagger-ui.html`, `/v3/api-docs` | Documentación (solo `dev` y `demo`). |

## 5.3 Ejemplos

Login:

```
POST /api/auth/login
Content-Type: application/json

{ "username": "farmaceutico", "password": "demo1234" }
```

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "expiraEn": "2026-11-12T22:30:00Z",
  "usuario": {
    "id": 2, "username": "farmaceutico", "nombreCompleto": "…", "rol": "FARMACEUTICO",
    "permisos": ["LECTURA", "CATALOGO_EDITAR", "STOCK_INGRESAR", "STOCK_BAJA_AJUSTE",
"TRANSFERENCIA_CREAR"]
  }
}
```

Transferencia:

```
POST /api/transferencias
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{ "areaId": 1, "observaciones": "Pedido de la mañana",
  "items": [ { "medicamentoId": 5, "cantidad": 150 } ] }
```

Respuesta 201 (el FEFO tomó dos lotes):

```
{
  "id": 42, "numero": "TR-000042", "fechaHora": "2026-11-12T09:15:00-03:00",
  "area": { "id": 1, "nombre": "Guardia" },
  "realizadaPor": { "id": 2, "nombreCompleto": "…" },
  "observaciones": "Pedido de la mañana",
  "detalle": [
    { "medicamentoId": 5, "medicamento": "Paracetamol comprimidos 500 mg", "loteId": 10, "numeroLote":
"L2301", "fechaVencimiento": "2026-12-01", "cantidad": 100 },
    { "medicamentoId": 5, "medicamento": "Paracetamol comprimidos 500 mg", "loteId": 14, "numeroLote":
"L2340", "fechaVencimiento": "2027-03-15", "cantidad": 50 }
  ]
}
```

## 5.4 Errores

Formato **RFC 9457 (Problem Details)** con la propiedad extra `codigo`:

```
{ "type": "about:blank", "title": "Datos inválidos", "status": 400,
  "detail": "Revisá los datos ingresados.", "codigo": "VALIDACION",
  "errors": { "fechaVencimiento": "La fecha de vencimiento no puede ser anterior a hoy." } }
```

| Caso | Status | `codigo` | Extra |
|---|---|---|---|
| Validación de campos | 400 | `VALIDACION` | `errors: {campo: mensaje}` |

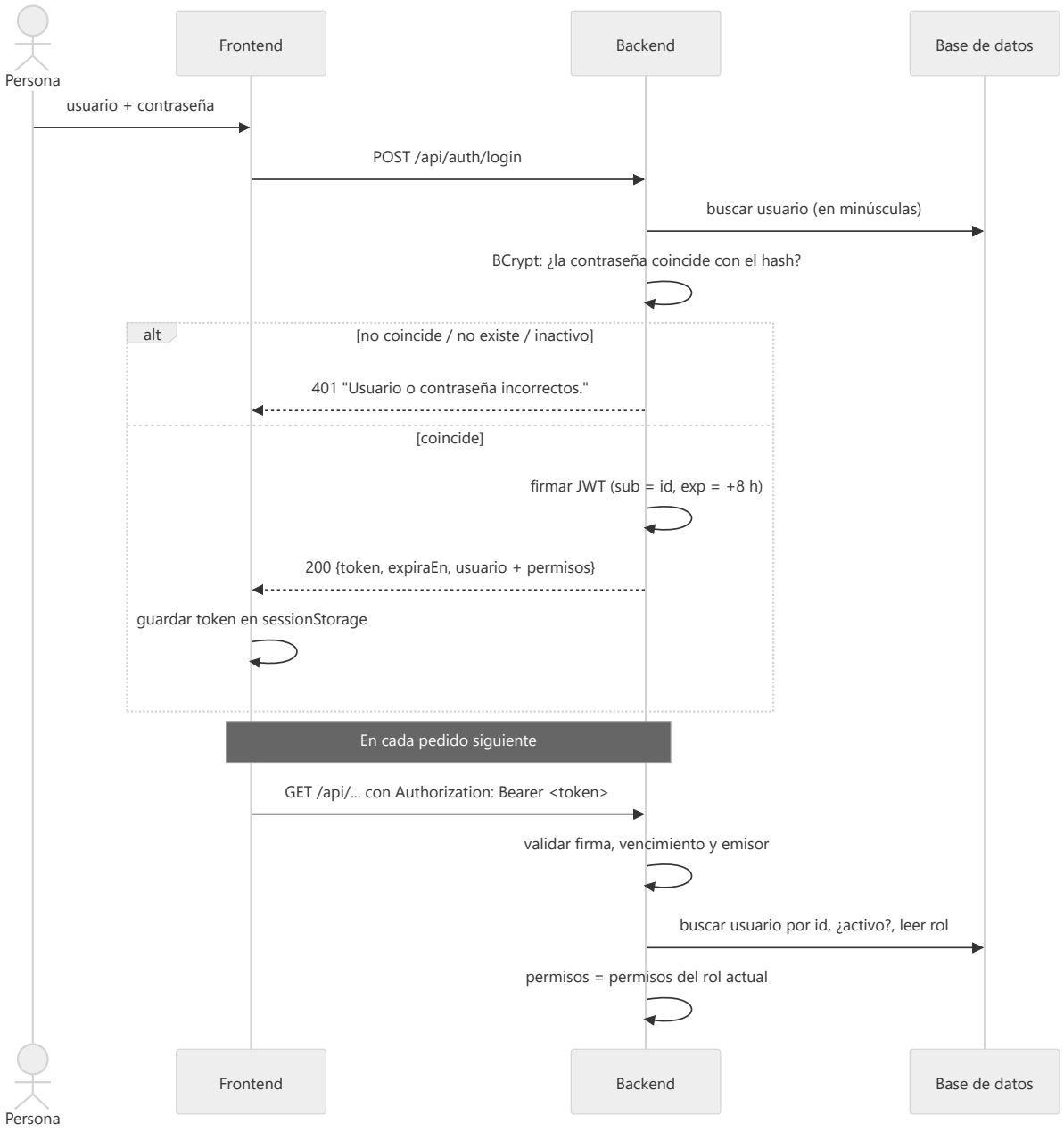
| Caso | Status | `codigo` | Extra |
|---|---|---|---|
| JSON mal formado o tipo inválido | 400 | `CUERPO_INVALIDO` | — |
| Parámetro u orden inválido | 400 | `PARAMETRO_INVALIDO`, `ORDEN_INVALIDO` | — |
| Sin sesión, token inválido o vencido, usuario inactivo | 401 | `NO_AUTENTICADO` | — |
| Sin permiso | 403 | `SIN_PERMISO` | — |
| No existe | 404 | `NO_ENCONTRADO` | — |
| Stock insuficiente | 409 | `STOCK_INSUFICIENTE` | `faltantes: [{medicamentoId, medicamento, disponible, solicitado}]` |
| Duplicados | 409 | `CODIGO_DUPLICADO`, `LOTE_DUPLICADO`, `CUIT_DUPLICADO`, `USERNAME_DUPLICADO`, `NOMBRE_DUPLICADO` | — |
| Reglas de negocio | 409 | `AREA_INACTIVA`, `PROVEEDOR_INACTIVO`, `MEDICAMENTO_INACTIVO`, `MEDICAMENTO_CON_STOCK`, `LOTE_NO_VENCIDO`, `LOTE_SIN_STOCK`, `ULTIMO_ADMIN`, `PROPIO_USUARIO` | — |
| Error inesperado | 500 | `ERROR_INTERNO` | `errorId` (también en el log del servidor) |

## 5.5 Formatos de exportación

| Formato | Detalle |
|---|---|
| CSV | UTF-8 con BOM (para que Excel muestre las tildes), separador `;`, fin de línea `\r\n`, fechas `dd/mm/aaaa HH:mm`. Valores que empiezan con `=`, `+`, `-`, `@`, tabulación o retorno de carro llevan `'` adelante (anti-inyección de fórmulas). |
| XLSX | Título, filtros aplicados, encabezado de color fijo al desplazarse, filtros de columna, anchos según el contenido, números y fechas como tipos reales. |
| PDF | Título, filtros, fecha de generación y tabla con la identidad del sistema. |

# 6. Seguridad

## 6.1 Autenticación (quién sos)




| Medida | Detalle | Dónde |
|---|---|---|
| Hash de contraseñas | BCrypt, factor 12 ($2^{12}$ rondas). Contraseña de 8 a 72 caracteres (BCrypt no admite más de 72 bytes). | `SeguridadConfig.passwordEncoder` |
| Mismo error en todos los casos | Usuario inexistente, inactivo o contraseña incorrecta → mismo 401 y mismo mensaje. `DaoAuthenticationProvider` calcula un hash aunque el usuario no exista, para que el tiempo de respuesta tampoco lo revele. | `AuthService`, `SeguridadConfig` |
| Token JWT | HS256 (HMAC con SHA-256). Claims: `iss` (farmacia-hospitalaria), `sub` (id del usuario), `iat`, `exp`. **Sin rol ni datos personales.** Dura 8 horas. | `TokenService` |

| Medida | Detalle | Dónde |
|---|---|---|
| Clave del token | `JWT_SECRET` por variable de entorno, mínimo 32 bytes. Si falta o es corta, la aplicación no arranca y el mensaje no muestra la clave. | `JwtProperties` |
| Validación del token | Firma, vencimiento (con el reloj de la aplicación y 60 s de tolerancia) y emisor. | `SeguridadConfig.jwtDecoder` |
| Usuario en cada pedido | Se busca por el `sub`; si no existe o está inactivo → 401. Los permisos son los del rol **actual** en la base. | `UsuarioJwtConverter` |
| Sin sesión del servidor | `STATELESS`. Sin cookies. | `SeguridadConfig` |
| Sin registro público | Solo `USUARIO_GESTIONAR` crea usuarios. | `UsuarioController` |
| Primer administrador | Si no hay ningún ADMIN al arrancar, se crea `admin` con `ADMIN_PASSWORD`. Si ya hay uno, la variable se ignora. | `AdminInicial` |

### Recuperación de contraseña (simulada)

- Se pide con el nombre de usuario. La respuesta es **la misma exista o no** el usuario.
- Código de 6 números generado con `SecureRandom`, guardado con **BCrypt** (nunca en texto), vence a los **15 minutos**, se anulan los pedidos anteriores.
- **5 intentos** fallidos y el código se anula aunque el siguiente sea el correcto. El contador se guarda aunque la operación falle (`noRollbackFor`) y se incrementa con bloqueo pesimista, para que dos intentos simultáneos no lo burlen.
- El envío pasa por la interfaz `NotificadorRecuperacion`. La implementación actual (`CorreoSimulado`) solo deja constancia en el log, **sin el código**. En los perfiles `demo` y `test` la respuesta incluye el código para mostrarlo en pantalla. Para enviar correos reales alcanza con otra implementación de la interfaz.

## 6.2 Autorización (qué podés hacer)

| Permiso | Habilita | ADMIN | FARMACÉUTICO | CONSULTA |
|---|---|---|---|---|
| `LECTURA` | Todos los `GET` salvo usuarios; reportes | ✓ | ✓ | ✓ |
| `CATALOGO_EDITAR` | Alta, edición y estado de proveedores, áreas y medicamentos | ✓ | ✓ | — |
| `STOCK_INGRESAR` | Ingreso de lotes | ✓ | ✓ | — |
| `STOCK_BAJA_AJUSTE` | Bajas por vencimiento y ajustes | ✓ | ✓ | — |
| `TRANSFERENCIA_CREAR` | Crear transferencias | ✓ | ✓ | — |
| `USUARIO_GESTIONAR` | Todo `/api/usuarios` | ✓ | — | — |

- El mapa rol → permisos está en **un solo lugar**: el enum `Rol`.
- Cada método de los controladores declara su permiso: `@PreAuthorize("hasAuthority('CATALOGO_EDITAR')")`.
- El frontend recibe los permisos en `/api/auth/me` y **oculta** lo que no corresponde (`puede(permiso)`), pero **la seguridad real está en el backend**: aunque alguien escriba la URL o modifique el navegador, el servidor responde 403.

- Reglas adicionales: nadie se desactiva a sí mismo (`PROPIO_USUARIO`); no se puede dejar el sistema sin administradores activos (`ULTIMO_ADMIN`).

## 6.3 Protección de datos

| Riesgo | Medida |
|---|---|
| Exponer el hash de la contraseña | Las respuestas son `record` DTO; ningún DTO tiene el hash. Prueba: `SeguridadIT.ningunaRespuestaExponeLaContrasenaNiElHash`. |
| Secretos en el repositorio | `JWT_SECRET`, `ADMIN_PASSWORD`, `DB_PASSWORD` solo por variables de entorno; `.env` está en `.gitignore`. |
| Datos sensibles en los logs | Nunca se registran el token, la contraseña ni el cuerpo del login. El código de recuperación tampoco. |
| Errores con información interna | El 500 responde solo "Ocurrió un error inesperado." con un `errorId`; el detalle queda en el log. `include-stacktrace: never`. |
| Endpoints de administración expuestos | Actuator solo publica `/actuator/health`, sin detalles. Swagger apagado en `prod`. |
| Base de datos accesible desde la red | Docker publica el puerto solo en `127.0.0.1`. |

## 6.4 Vulnerabilidades web consideradas (OWASP Top 10)

| Riesgo | Cómo se mitiga |
|---|---|
| **A01 Control de acceso roto** | Identidad solo del JWT validado (una cabecera `X-User-Id` se ignora: `SeguridadIT`); permisos en cada método; el usuario de una transferencia sale del token, nunca del cuerpo del pedido. |
| **A02 Fallas criptográficas** | BCrypt; HMAC-SHA256 con clave de 32+ bytes; códigos de recuperación hasheados. |
| **A03 Inyección SQL** | Consultas parametrizadas (JPA y `JdbcClient` con parámetros nombrados); los campos de orden salen de una lista blanca. |
| **A03 Inyección de fórmulas (CSV)** | Valores que empiezan con `=`, `+`, `-`, `@`, tab o CR llevan `'` adelante (`CsvTest`). |
| **A03 XSS (Cross-Site Scripting)** | React escapa todo el texto; ESLint prohíbe `innerHTML` y `dangerouslySetInnerHTML`; CSP estricta en nginx. Prueba: un medicamento llamado `<img src=x onerror=alert(1)>` se ve como texto (`seguridad.test.tsx`). |
| **A04 Diseño inseguro** | "Todo o nada" y bloqueos en las transferencias; invariantes en entidad y base. |
| **A05 Configuración insegura** | La aplicación no arranca sin clave, con clave corta ni con `demo` y `prod` juntos (`ConfiguracionTest`). Imágenes Docker sin root. |
| **A07 Fallas de autenticación** | Mismo mensaje en todos los errores de login; tokens con vencimiento; usuario desactivado → 401 inmediato. |
| **CSRF** | No aplica: la API no usa cookies, el token se manda a mano en una cabecera que el navegador no agrega solo. Por eso CSRF está deshabilitado (decisión documentada). |
| **CORS** | Vacío por defecto (mismo origen detrás del proxy). Si se configura, nunca `*`. |

## 6.5 Seguridad en el frontend

| Medida | Detalle |
|---|---|
| Token en `sessionStorage` | Se borra al cerrar la pestaña (útil en PC compartidas). Nunca va en la URL ni en los logs. |
| 401 | Borra la sesión y lleva al ingreso con "Tu sesión expiró". |
| 403 | Muestra "No tenés permiso para esta acción." y refresca los permisos. |
| Aviso de vencimiento | 5 minutos antes de que venza el token. |
| Salir | Borra la sesión y la caché de datos; la siguiente persona empieza en el panel. |
| Redirecciones | Después del ingreso solo se vuelve a rutas internas (no a sitios externos). |
| Cabeceras de nginx | `Content-Security-Policy` estricta (solo recursos del propio sitio, `frame-ancestors 'none'` para que nadie lo meta en un iframe), `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer` y `Permissions-Policy` que apaga cámara, micrófono, ubicación, pagos y USB (archivo `frontend/nginx-cabeceras.conf`). |

## 6.6 Pruebas de seguridad

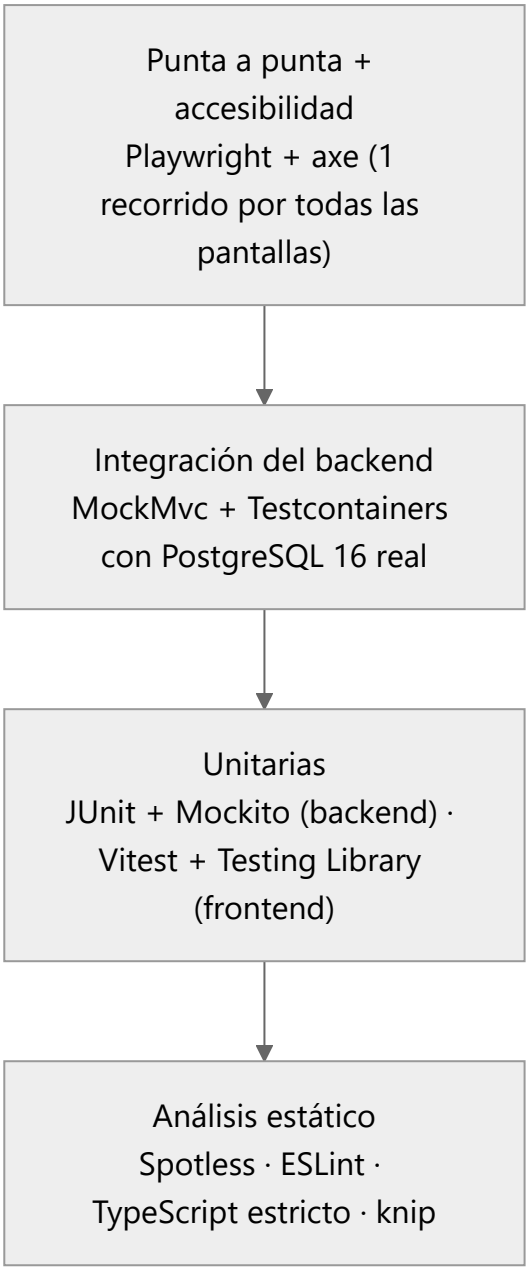
`SeguridadIT` verifica: 401 sin token (en JSON, sin redirigir); 401 con firma inválida, token vencido y usuario desactivado; 403 de CONSULTA creando un medicamento y de FARMACÉUTICO en usuarios; que la cabecera `X-User-Id` no cambia nada; mismo 401 para contraseña incorrecta y usuario inexistente; y que ninguna respuesta contiene `password` ni `hash`. `ConfiguracionTest` verifica el arranque seguro y `RecuperacionIT` / `RecuperacionPasswordTest` la recuperación.

# 7. Plan de pruebas

## 7.1 Objetivo

Demostrar que el sistema cumple los requisitos y las reglas de negocio, que ninguno de los problemas del prototipo se repite y que un cambio futuro no rompe lo que ya funciona.

## 7.2 Niveles y herramientas




Pirámide de pruebas: muchas pruebas chicas y rápidas abajo, pocas grandes arriba.

| Nivel | Cantidad | Cómo se corre | Tiempo aprox. |
|---|---|---|---|
| Backend (unitarias `*Test` + integración `*IT`) | 94 | `cd backend && ./mvnw verify` (necesita Docker) | 1–3 min |
| Frontend (Vitest) | 97 | `cd frontend && npm test` | 15 s |
| Punta a punta + axe | 1 recorrido, 16 pantallas | `cd frontend && npm run capturas` (con la app levantada) | 25 s |

**Reloj fijo:** en las pruebas del backend, "hoy" es una fecha fija (bean `Clock` `@Primary`), así los casos de vencimiento dan siempre el mismo resultado.

**Base real:** las pruebas de integración usan un PostgreSQL 16 en un contenedor (Testcontainers), igual al de producción, para probar las restricciones `CHECK`, los bloqueos y el SQL de verdad. Un solo contenedor y un solo contexto de Spring para todas (`IntegracionBase`).

## 7.3 Pruebas del backend

| Clase | Tipo | Qué verifica |
|---|---|---|
| `TransferenciaServiceTest` | Unitaria | FEFO con 3 lotes; desempate por ingreso e id; todo o nada; todos los faltantes; ítems repetidos se suman; cantidad 0 o negativa; lote vencido ayer ignorado y lote que vence hoy usado; área y medicamento inactivos; usuario del contexto de seguridad. |
| `AlertaServiceTest` | Unitaria | Los cuatro tipos de alerta y los bordes: vence hoy, vence en N, vence en N+1, venció ayer, medicamento inactivo con lote vencido, stock igual al mínimo, mínimo 0 con stock 0, lotes en 0. |
| `LoteTest` | Unitaria | `descontar` de más lanza excepción; ajuste fuera de rango también. |
| `CuitValidatorTest` | Unitaria | Válidos: `20-12345678-6`, `30-71234567-1`, `27-33333333-9`. Inválidos: `20-12345678-5`, `20-99999999-0`, `2012345678`, `20-1234567A-6`. |
| `IngresoServiceTest` | Unitaria | Vencimiento de ayer rechazado; de hoy aceptado; lote duplicado 409. |
| `ProveedorServiceTest` | Unitaria | Baja con lotes con stock permitida y lotes disponibles; proveedor inactivo rechazado en ingresos; ninguna baja borra filas. |
| `CsvTest` | Unitaria | Inyección de fórmulas, comillas y `;` en los valores. |
| `RecuperacionPasswordTest` | Unitaria | Vencimiento, intentos máximos, cierre del pedido. |
| `ConfiguracionTest` | Arranque | No arranca sin `JWT_SECRET`, con clave < 32 bytes ni con `demo` + `prod`. |
| `MedicamentoRepositoryIT` | Integración | Stock excluye vencidos; filtro por estado; orden por stock y próximo vencimiento; total con paginación. |
| `FlujoCompletoIT` | Integración | Login → área, proveedor, medicamento → dos lotes → transferencia → stock, detalle FEFO, movimientos; suma de movimientos = stock; exportación. |
| `TransferenciaConcurrenciaIT` | Integración | Dos hilos piden 60 de un lote de 100 a la vez: uno 201, otro 409, stock final 40. Con dos medicamentos en orden inverso: sin deadlock. |

| Clase | Tipo | Qué verifica |
|---|---|---|
| `SeguridadIT` | Integración | `sinTokenDevuelve401EnJsonSinRedirigir`, `tokenConFirmaInvalidaDevuelve401`, `tokenVencidoDevuelve401`, `tokenDeUnUsuarioDesactivadoDevuelve401` (crea y desactiva un usuario por la API), `consultaNoPuedeCrearMedicamentos`, `farmaceuticoNoPuedeGestionarUsuarios`, `laCabeceraXUserIdNoCambiaLaIdentidad`, `contrasenaIncorrectaYUsuarioInexistenteRespondenIgual`, `ningunaRespuestaExponeLaContrasenaNiElHash`. |
| `ErroresIT` | Integración | 400 con los campos con error, JSON mal formado sin detalles internos, orden no permitido, 404, duplicado 409 con `codigo` y 500 con `errorId` sin stacktrace. |
| `RecuperacionIT` | Integración | Flujo completo de recuperación; misma respuesta exista o no el usuario; 5 intentos. |
| `ReportesIT` | Integración | Stock de medicamentos en CSV (respetando filtros), Excel y PDF; lotes vencidos y por vencer en CSV y PDF; comprobante y reporte de transferencias en PDF; el rol CONSULTA puede exportar y sin token no. |

## 7.4 Pruebas del frontend

| Archivo | Qué verifica |
|---|---|
| `cliente.test.ts` | Cabecera Bearer; 401 cierra la sesión (menos en el login); 403; 5xx con `errorId`; error de red; tiempo máximo 15 s; 204. |
| `consultas.test.ts` | Reintento solo ante red caída o 5xx, una sola vez. |
| `SesionProvider.test.tsx` | Validación con `/me` al iniciar; aviso 5 minutos antes del vencimiento. |
| `RutaProtegida.test.tsx` | Sin sesión → ingreso y vuelta a la ruta pedida; sin permiso → "Sin permiso". |
| `PaginaIngreso.test.tsx` | Foco, "Ingresando…", mensaje de credenciales, sesión expirada. |
| `PaginaRecuperacion.test.tsx` | Los dos pasos de la recuperación y sus errores. |
| `NuevaTransferencia.test.tsx` | Agregar, sumar repetidos, validar cantidades, resumen, 409 con faltantes por línea, borrador. |
| `PaginaReportes.test.tsx` | Ver en pantalla y descargas con los filtros. |
| `Tabla`, `Modal`, `Paginador`, `Estados`, `Insignia` | Componentes reutilizables: orden con `aria-sort`, foco del modal, estados de carga, vacío y error, insignias con texto. |
| `seguridad.test.tsx` | Un texto con HTML malicioso se muestra como texto. |
| `useFormulario`, `useFiltrosUrl`, `useDebounce` | Validación al salir y al enviar; filtros en la URL; espera de 300 ms en la búsqueda. |
| `cuit`, `fechas`, `numeros`, `estadoStock`, `validaciones` | Utilidades: los mismos vectores de CUIT que el backend; fechas sin corrimiento de zona horaria; formato `es-AR`. |

## 7.5 Prueba punta a punta y accesibilidad

`frontend/e2e/capturas.spec.ts` recorre como administrador: recuperar contraseña, ingreso, panel, medicamentos, detalle, nuevo medicamento, ingreso de mercadería, nueva transferencia (con un ítem), historial, comprobante, lotes vencidos, proveedores, áreas, usuarios, mi cuenta y reportes. En cada pantalla espera que

terminen las cargas y las animaciones, corre **axe-core** con las reglas WCAG 2.0/2.1 A y AA y **falla si hay violaciones serias o críticas**; después guarda la captura en `docs/capturas/`. También falla si aparece algún error en la consola del navegador.

## 7.6 Matriz de trazabilidad

| Requisito | Pruebas |
|---|---|
| RF01 Iniciar sesión | `SeguridadIT`, `PaginaIngreso.test.tsx`, e2e |
| RF04 Recuperar contraseña | `RecuperacionIT`, `RecuperacionPasswordTest`, `PaginaRecuperacion.test.tsx` |
| RF06–RF10 Usuarios | `SeguridadIT` (alta y desactivación por la API, permisos), e2e (pantalla Usuarios) |
| RF11 Medicamentos | `MedicamentoRepositoryIT`, `ErroresIT`, e2e |
| RF13 Proveedores | `ProveedorServiceTest`, `CuitValidatorTest`, `cuit.test.ts` |
| RF15 Ingreso | `IngresoServiceTest`, `FlujoCompletoIT` |
| RF16 Transferir (FEFO) | `TransferenciaServiceTest`, `FlujoCompletoIT`, `TransferenciaConcurrenciaIT`, `NuevaTransferencia.test.tsx` |
| RF17 Comprobante | `ReportesIT` (PDF), e2e |
| RF20–RF21 Baja y ajuste | `LoteTest` (reglas de la entidad); prueba manual de aceptación 6. *Pendiente: una prueba de integración de los endpoints `/baja` y `/ajuste`.* |
| RF22 Movimientos | `FlujoCompletoIT` |
| RF23–RF24 Alertas y panel | `AlertaServiceTest`, e2e |
| RF25–RF27 Reportes | `ReportesIT`, `CsvTest`, `PaginaReportes.test.tsx` |
| RNF01–RNF04 Seguridad | `SeguridadIT`, `ConfiguracionTest`, `seguridad.test.tsx` |
| RNF05 Stock nunca negativo | `LoteTest`, `TransferenciaServiceTest`, `TransferenciaConcurrenciaIT` |
| RNF07 Trazabilidad | `FlujoCompletoIT` |
| RNF08 Zona horaria | `AlertaServiceTest`, `fechas.test.ts` |
| RNF10 Accesibilidad | e2e con axe |

## 7.7 Pruebas manuales de aceptación

Para la demostración o una validación con usuarios reales (con el perfil `demo`):

| # | Paso | Resultado esperado |
|---|---|---|
| 1 | Ingresar como `consulta`. | El menú no muestra Nueva transferencia, Ingreso de mercadería ni Usuarios. No hay botones de alta ni de baja. |
| 2 | Como `consulta`, escribir `/usuarios` en la barra de direcciones. | "No tenés permiso para ver esta página." |
| 3 | Ingresar como `farmaceutico`, registrar un lote que vence en 10 días. | Aviso de que vence pronto; se registra igual. El panel suma uno a "Por vencer". |

| # | Paso | Resultado esperado |
|---|---|---|
| 4 | Transferir 150 unidades de un medicamento con dos lotes (100 que vence antes y 200 que vence después). | El comprobante muestra 100 del primero y 50 del segundo. |
| 5 | Pedir más de lo disponible. | Se marca la línea con "Disponible: X (pediste Y)"; nada se descuenta. |
| 6 | Dar de baja un lote vencido. | Queda en 0, aparece el movimiento "Baja por vencimiento". |
| 7 | Intentar dar de baja un medicamento con stock. | El diálogo muestra que no se puede porque tiene stock. |
| 8 | Descargar el reporte de transferencias en Excel y en PDF. | Los dos archivos abren y muestran las mismas filas. |
| 9 | Como `admin`, desactivar a `farmaceutico` mientras está con sesión abierta en otra ventana. | En su próxima acción, `farmaceutico` vuelve al ingreso. |
| 10 | Recuperar la contraseña de `consulta`. | Se muestra el código (modo demo); con el código se cambia la contraseña. |

## 7.8 Criterios de aceptación de la entrega

- `./mvnw verify` en verde, sin pruebas salteadas, con el formato aplicado.
- `npm run lint`, `npm run typecheck`, `npm test` y `npm run build` en verde.
- `npm run capturas` sin violaciones de accesibilidad serias o críticas.
- `docker compose up --build` desde cero funciona y `/actuator/health` responde `UP`.

# 8. Instalación y despliegue (manual técnico)

## 8.1 Requisitos

| Para | Necesitás |
|---|---|
| Levantar el sistema | **Docker Desktop** (Windows/macOS) o Docker Engine + Compose (Linux), corriendo. |
| Desarrollar el backend o correr sus pruebas fuera de Docker | **JDK 21**. Maven **no** hace falta: el proyecto trae el *wrapper* (`mvnw` / `mvnw.cmd`). |
| Desarrollar el frontend | **Node.js 24 LTS** (mínimo 22.22) y npm. |
| Clonar el repositorio | Git. |

Hardware orientativo para un servidor del hospital: 2 núcleos, 4 GB de RAM, 10 GB de disco.

## 8.2 Obtener el código

```
git clone [COMPLETAR: URL del repositorio]
cd Farmacia
```

## 8.3 Configurar las variables de entorno

```
cp .env.example .env          # PowerShell: Copy-Item .env.example .env
```

Editá `.env` y completá:

| Variable | Qué poner |
|---|---|
| `DB_PASSWORD` | Una contraseña para la base de datos. |
| `JWT_SECRET` | Una clave aleatoria de al menos 32 bytes (ver abajo cómo generarla). |
| `ADMIN_PASSWORD` | La contraseña del usuario `admin` (mínimo 8 caracteres). Solo se usa la primera vez. |
| `DEMO_PASSWORD` | Contraseña de `farmaceutico` y `consulta` en el modo demo (por defecto `demo1234`). |
| `SPRING_PROFILES_ACTIVE` | `demo` (con datos de ejemplo) o `prod` (vacío, para uso real). |
| `DB_PUERTO` | Cambialo (por ejemplo a 5433) si ya tenés otro PostgreSQL en el 5432. |
| `CORS_ORIGENES` | Vacío si el frontend va detrás del mismo proxy (lo normal). |

Generar `JWT_SECRET`:

```
openssl rand -base64 48
```

```
$b = New-Object byte[] 48; [Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b);
[Convert]::ToBase64String($b)
```

> El archivo `.env` tiene secretos: **nunca** se sube al repositorio (está en `.gitignore`).

## 8.4 Levantar la base y el backend

```
docker compose up --build
```

- La primera vez descarga las imágenes y compila el backend (varios minutos).
- Cuando aparece `Started FarmaciaApplication`, la API está en http://localhost:8080.
- Verificación: http://localhost:8080/actuator/health → `{"status":"UP"}`.
- Swagger (perfil demo): http://localhost:8080/swagger-ui.html.

Para dejarlo corriendo en segundo plano: `docker compose up -d --build`. Para ver los logs: `docker compose logs -f api`. Para detenerlo: `docker compose down` (los datos quedan en el volumen). Para **borrar también los datos**: `docker compose down -v`.

## 8.5 Levantar el frontend

### Desarrollo

```
cd frontend
npm install
npm run dev            # http://localhost:5173
```

Vite reenvía `/api` a `http://localhost:8080`. Si el backend está en otro puerto:

```
$env:API_DESTINO = "http://localhost:8090"; npm run dev
```

### Producción

```
cd frontend
npm run build          # genera frontend/dist/
docker build -t farmacia-web .
```

La imagen (`nginxinc/nginx-unprivileged`) sirve `dist/` en el puerto 8080 del contenedor, reenvía `/api/` al servicio `api` y agrega las cabeceras de seguridad. Para integrarla al compose se agrega un servicio `web` en la misma red que `api`, por ejemplo:

```
  web:
    build: ./frontend
    depends_on:
      - api
    ports:
      - "8081:8080"
```

y se entra por http://localhost:8081. Para publicar en la red del hospital conviene poner HTTPS (certificado del hospital) en un proxy delante de `web`.

## 8.6 Usuarios iniciales

| Usuario | Contraseña | Rol | Cuándo existe |
|---|---|---|---|
| `admin` | la de `ADMIN_PASSWORD` | ADMIN | Siempre (se crea si no hay ningún ADMIN). |

| Usuario | Contraseña | Rol | Cuándo existe |
|---|---|---|---|
| `farmaceutico` | `DEMO_PASSWORD` | FARMACEUTICO | Solo con el perfil `demo`. |
| `consulta` | `DEMO_PASSWORD` | CONSULTA | Solo con el perfil `demo`. |

En producción, el `admin` crea los usuarios reales desde la pantalla **Usuarios** y conviene que cambie su contraseña desde **Mi cuenta**.

## 8.7 Desarrollo del backend sin Docker para la API

```
docker compose up -d db
cd backend
DB_PASSWORD=<la del .env> JWT_SECRET=... ADMIN_PASSWORD=... SPRING_PROFILES_ACTIVE=dev,demo ./mvnw spring-boot:run
```

```
docker compose up -d db
cd backend
$env:DB_PASSWORD="<la del .env>"; $env:JWT_SECRET="..."; $env:ADMIN_PASSWORD="...";
$env:SPRING_PROFILES_ACTIVE="dev,demo"
.\mvnw.cmd spring-boot:run
```

## 8.8 Backend simulado (solo frontend)

Para trabajar en el frontend sin levantar Java ni la base:

```
cd frontend
PUERTO_MOCK=8090 npm run mock                       # datos en memoria; usuarios
admin/farmaceutico/consulta, contraseña demo1234
API_DESTINO=http://localhost:8090 npm run dev       # en otra terminal
```

## 8.9 Comandos de calidad

| Dónde | Comando | Qué hace |
|---|---|---|
| `backend/` | `./mvnw verify` | Formato + pruebas unitarias + integración (Docker corriendo). Reporte de cobertura en `target/site/jacoco/index.html`. |
| `backend/` | `./mvnw spotless:apply` | Aplica el formato (antes de cada commit). |
| `frontend/` | `npm run lint` | ESLint sin advertencias. |
| `frontend/` | `npm run typecheck` | Verificación de tipos. |
| `frontend/` | `npm test` | Pruebas con Vitest. |
| `frontend/` | `npm run knip` | Código y dependencias sin usar. |
| `frontend/` | `npm run capturas` | Recorrido e2e + accesibilidad + capturas (la primera vez: `npx playwright install chromium`). |
| `frontend/` | `npm run api:tipos` | Regenera los tipos desde el OpenAPI del backend levantado. |

## 8.10 Copias de seguridad

```
# Respaldo
docker compose exec db pg_dump -U farmacia farmacia > respaldo_$(date +%F).sql
# Restauración (con la base vacía)
docker compose exec -T db psql -U farmacia farmacia < respaldo_2026-11-12.sql
```

Se recomienda un respaldo diario automático (tarea programada de Windows o `cron`) guardado fuera del servidor.

## 8.11 Problemas frecuentes

| Síntoma | Causa | Solución |
|---|---|---|
| `Definí DB_PASSWORD en .env` | Falta el `.env` o la variable. | Copiá `.env.example` a `.env` y completalo. |
| La API no arranca: mensaje sobre `JWT_SECRET` | Falta la clave o tiene menos de 32 bytes. | Generá una con el comando de 8.3. |
| `port is already allocated` en 5432 | Hay otro PostgreSQL en esa máquina. | Poné `DB_PUERTO=5433` en `.env`. |
| `port is already allocated` en 8080 | Otro programa usa el 8080. | Cerralo o cambiá el puerto publicado en `docker-compose.yml`. |
| `/bin/sh^M: bad interpreter` al compilar | `mvnw` con fines de línea de Windows. | El repo trae `.gitattributes` que lo evita; volvé a clonar o `git checkout -- backend/mvnw`. |
| Las pruebas del backend fallan con error de Docker | Testcontainers necesita Docker. | Abrí Docker Desktop y esperá a que diga "running". |
| El frontend muestra "No se pudo conectar con el servidor." | El backend no está levantado o el proxy apunta a otro puerto. | Verificá `/actuator/health` y la variable `API_DESTINO`. |
| Cambié `ADMIN_PASSWORD` y no anda | Solo se usa si no existe ningún ADMIN. | Cambiá la contraseña desde **Mi cuenta** o borrá los datos con `docker compose down -v` (pierde todo). |

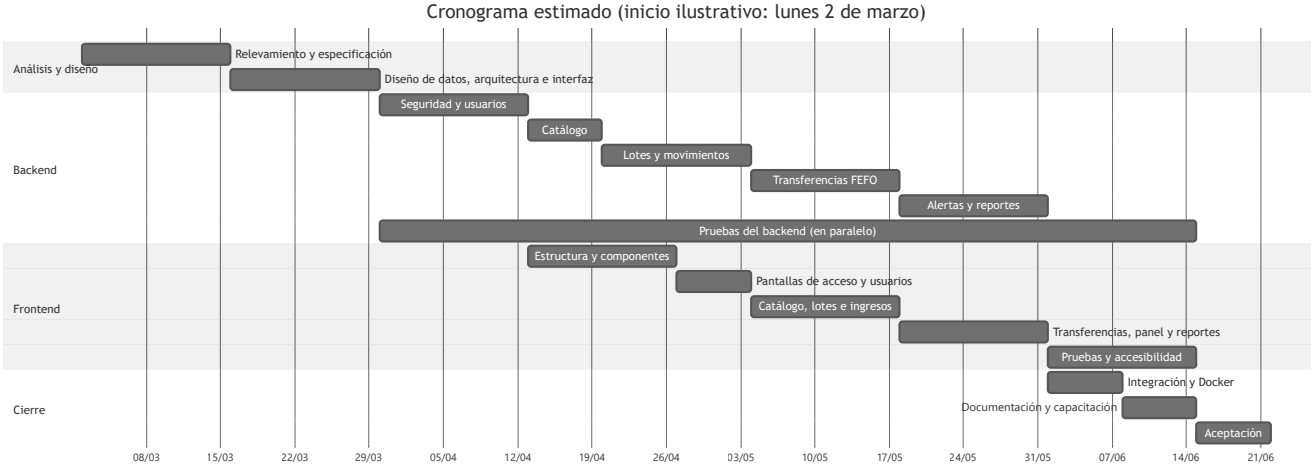
# 9. Planificación, riesgos y presupuesto

## 9.1 Etapas y estimación del sistema original

| # | Etapa | Horas |
|---|---|---|
| 1 | Relevamiento, análisis y especificación funcional | 24 |
| 2 | Diseño: modelo de datos, arquitectura y diseño visual | 28 |
| 3 | Backend: seguridad (JWT, roles, permisos), usuarios y recuperación de contraseña | 32 |
| 4 | Backend: catálogo de medicamentos, proveedores y áreas | 24 |
| 5 | Backend: lotes, ingresos, bajas por vencimiento, ajustes y movimientos | 28 |
| 6 | Backend: transferencias FEFO con bloqueo y control de concurrencia | 32 |
| 7 | Backend: alertas, panel y reportes en CSV, Excel y PDF | 28 |
| 8 | Backend: pruebas unitarias y de integración (Testcontainers) | 32 |
| 9 | Frontend: estructura, componentes reutilizables y diseño visual | 36 |
| 10 | Frontend: ingreso, recuperación de contraseña, mi cuenta y usuarios | 20 |
| 11 | Frontend: medicamentos, proveedores, áreas, lotes e ingresos | 40 |
| 12 | Frontend: transferencias, comprobante, panel y reportes | 40 |
| 13 | Frontend: pruebas y accesibilidad (WCAG AA) | 24 |
| 14 | Integración, Docker y puesta en marcha | 16 |
| 15 | Documentación y capacitación de usuarios | 16 |
| 16 | Gestión del proyecto y pruebas de aceptación | 24 |
| | **Total** | **444** |

## 9.2 Cronograma estimado

Suponiendo **una persona a 20 horas semanales** (≈ 22 semanas). Las etapas de backend y frontend se pueden solapar porque el contrato de la API se define antes.



Hitos: (1) especificación aprobada; (2) API de seguridad y catálogo funcionando; (3) transferencias FEFO con prueba de concurrencia en verde; (4) todas las pantallas; (5) aceptación.

## 9.3 Riesgos

| Riesgo | Probabilidad | Impacto | Mitigación |
|---|---|---|---|
| Dos transferencias simultáneas dejan stock negativo | Media | Alto | Bloqueo pesimista ordenado + `CHECK` en la base + prueba de concurrencia. |
| Lotes vistos como vencidos por la zona horaria | Alta (contenedores en UTC) | Medio | `Clock` con zona de Buenos Aires; fechas del front sin `new Date`. |
| Cambios en Spring Boot 4 respecto de la versión 3 | Media | Medio | Verificar cada API contra la guía oficial de migración. |
| Diferencias entre el contrato y lo implementado | Media | Medio | Tipos del frontend generados del OpenAPI del backend. |
| Pérdida de datos | Baja | Alto | Bajas lógicas, sin cascadas, respaldos diarios (`pg_dump`). |
| Usuarios con poca experiencia informática | Media | Medio | Interfaz simple, mensajes claros, confirmaciones, manual de usuario y capacitación. |
| Inflación que desactualiza el presupuesto | Alta | Medio | Valor hora con fecha; actualizar antes de presentar. |
| Falta de correo institucional para la recuperación | Alta | Bajo | Recuperación simulada + restablecimiento por el administrador; interfaz lista para un envío real. |

## 9.4 Valor hora

**$25.000 por hora** (pesos argentinos, noviembre de 2026):

1. Sueldo de referencia: mediana de un desarrollador semi senior que cobra en pesos, $2.500.000 por mes (encuesta de sueldos de Sysarmy 2026.1).
2. Por hora como empleado: $2.500.000 / 160 h ≈ $15.600.
3. Ajuste por trabajo independiente (× 1,6): sin aguinaldo ni vacaciones pagas, con monotributo, equipo propio y meses sin trabajo. $15.600 × 1,6 ≈ **$25.000**.

## 9.5 Presupuesto

| Concepto | Horas | Costo |
|---|---|---|
| Sistema original | 444 | $11.100.000 |
| Mejora: órdenes de compra y reposición automática | 128 | $3.200.000 |
| **Total** | **572** | **$14.300.000** |

Desglose de la mejora:

| Tarea | Horas |
|---|---|
| Análisis, casos de uso y diseño del modelo (migración `V4`) | 10 |
| Backend: entidades, servicios y estados de la orden | 18 |
| Backend: cálculo de sugerencias por consumo promedio | 10 |

| Tarea | Horas |
|---|---|
| Backend: vínculo de los ingresos con la orden y recepción parcial | 8 |
| Backend: PDF de la orden y envío por correo (SMTP) | 10 |
| Backend: permisos y pruebas de integración | 14 |
| Frontend: listado de órdenes con filtros | 8 |
| Frontend: armado y edición de la orden con las sugerencias | 14 |
| Frontend: aprobación, envío y recepción | 8 |
| Frontend: reportes nuevos, pruebas y accesibilidad | 8 |
| Documentación, pruebas de aceptación y capacitación | 8 |
| Subtotal | 116 |
| Contingencia (10 %) por la integración con el correo del hospital | 12 |
| **Total de la mejora** | **128** |

### Condiciones

- Todo el software es libre y gratuito (Java, Spring Boot, PostgreSQL, React, Docker, OpenPDF, fastexcel): **sin costo de licencias**.
- No incluye hardware ni hosting: corre con Docker en un servidor del hospital.
- La mejora necesita una cuenta de correo SMTP del hospital.
- Forma de pago sugerida: 40 % al inicio, 30 % al entregar el backend y 30 % a la aceptación final.
- Validez: 30 días (por la inflación, el valor hora debe actualizarse después).

### Costos de operación (orientativos, a cargo del hospital)

| Concepto | Comentario |
|---|---|
| Servidor | Puede ser una máquina existente del hospital (2 núcleos, 4 GB RAM). |
| Respaldos | Disco externo o almacenamiento de red del hospital. |
| Mantenimiento correctivo y evolutivo | A convenir, por hora al mismo valor. |

## 9.6 Detalle de la mejora propuesta

Ver el informe final, sección 12, y `docs/presupuesto.md`. Resumen: sugerencia automática de compras según el consumo de los últimos 90 días, circuito de aprobación (Borrador → Aprobada → Enviada → Recibida / Anulada), orden en PDF enviada por correo, recepción vinculada a los ingresos y reportes de órdenes pendientes y cumplimiento de proveedores. Agrega las tablas `orden_compra` y `detalle_orden_compra` y una FK opcional en `lote`, manteniendo la 3FN.

# 10. Glosario

Términos del negocio y técnicos que aparecen en la documentación. Para explicaciones más largas, desde cero, ver `../estudio/16-glosario.md` *(en «Carpeta de estudio»)*.

## Del negocio

| Término | Significado |
|---|---|
| **Ajuste** | Corrección de la cantidad de un lote cuando el conteo físico no coincide (rotura, pérdida, error). Requiere motivo. |
| **Área** | Servicio del hospital que recibe medicamentos (Guardia, UTI, Pediatría…). |
| **Baja lógica** | Marcar un registro como inactivo (`activo = false`) en lugar de borrarlo. Se puede reactivar y no se pierde el historial. |
| **Baja por vencimiento** | Retiro del stock de un lote vencido. Lo deja en 0 y registra el motivo. |
| **Comprobante** | Documento de una transferencia (`TR-000123`) con el detalle por lote y lugar para firmas. |
| **CUIT** | Clave Única de Identificación Tributaria (11 dígitos, el último es verificador). |
| **Egreso** | Salida de stock de un lote por una transferencia. |
| **FEFO** | *First Expired, First Out*: sale primero lo que vence primero. |
| **FIFO** | *First In, First Out*: sale primero lo que entró primero. No se usa: con medicamentos puede dejar vencer un lote. |
| **Ingreso de mercadería** | Registro de un lote recibido de un proveedor. |
| **Lote** | Partida de un medicamento con un número y una fecha de vencimiento. Unidad de trazabilidad. |
| **Movimiento de stock** | Registro de cada cambio de cantidad de un lote: ingreso, egreso, baja o ajuste. |
| **Por vencer** | Lote con stock que vence entre hoy y dentro de 30 días. |
| **Presentación** | Forma y concentración del medicamento (comprimidos 500 mg, frasco ampolla 1 g). |
| **Proveedor** | Droguería o laboratorio que entrega la mercadería. |
| **Stock bajo** | Medicamento con stock mayor a 0 y menor o igual a su stock mínimo. |
| **Stock mínimo** | Cantidad por debajo de la cual hay que reponer. |
| **Stock total** | Suma de las cantidades de los lotes no vencidos de un medicamento. |
| **Transferencia** | Entrega de medicamentos de la farmacia a un área. |
| **Trazabilidad** | Poder reconstruir el recorrido de cada unidad: de qué proveedor y lote vino, a qué área fue, quién lo registró y cuándo. |
| **Vencido** | Lote cuya fecha de vencimiento es anterior a hoy. |

## Técnicos

| Término | Significado |
|---|---|
| **API REST** | Interfaz para que dos programas se comuniquen por HTTP usando URLs para los recursos y métodos (GET, POST, PUT, PATCH) para las operaciones. |
| **BCrypt** | Algoritmo para guardar contraseñas como un hash lento e irreversible. |
| **Bloqueo pesimista** | Reservar filas de la base (`SELECT … FOR UPDATE`) para que nadie más las modifique hasta terminar la transacción. |
| **Clave foránea (FK)** | Columna que apunta a la clave primaria de otra tabla. |
| **Clave primaria (PK)** | Columna que identifica de forma única cada fila. |
| **Contenedor (Docker)** | Paquete con un programa y todo lo que necesita para correr, aislado del resto del sistema. |
| **CORS** | Regla del navegador que impide que una página llame a un servidor de otro origen si este no lo autoriza. |
| **CSP** | *Content Security Policy*: cabecera que le dice al navegador de dónde puede cargar scripts y estilos. |
| **CSRF** | Ataque que aprovecha que el navegador manda cookies solo. No aplica a esta API (usa tokens en cabeceras). |
| **Deadlock** | Dos transacciones que se esperan mutuamente y quedan trabadas. |
| **DTO** | Objeto con los datos que entran o salen por la API. |
| **Endpoint** | Una ruta de la API con un método (por ejemplo `POST /api/transferencias`). |
| **Entidad** | Clase Java que representa una tabla. |
| **Flyway** | Herramienta que aplica los scripts de la base en orden y registra cuáles ya se aplicaron. |
| **Hash** | Resultado de una función que transforma un dato en otro de forma irreversible. |
| **JPA / Hibernate** | Estándar y biblioteca para trabajar con tablas como si fueran objetos Java. |
| **JWT** | *JSON Web Token*: token firmado que identifica al usuario en cada pedido. |
| **Migración** | Script versionado que cambia el esquema de la base (`V1__esquema.sql`). |
| **Normalización (3FN)** | Organización de las tablas para evitar datos repetidos y contradicciones. |
| **OpenAPI / Swagger** | Formato estándar para describir una API y herramienta para probarla desde el navegador. |
| **ProblemDetail** | Formato estándar de errores HTTP (RFC 9457). |
| **Perfil (Spring)** | Conjunto de configuración que se activa según el entorno (`dev`, `demo`, `prod`). |
| **SPA** | *Single Page Application*: la página se carga una vez y después cambia sin recargar. |
| **Testcontainers** | Biblioteca que levanta una base real en Docker durante las pruebas. |
| **Transacción** | Grupo de operaciones que se aplican todas o ninguna. |
| **WCAG** | Pautas internacionales de accesibilidad web. |